

CA' FOSCARI UNIVERSITY OF VENICE

Bachelor's Degree in Computer Science

ADVANCED AND DISTRIBUTED ALGORITHMS

Author

Giovanni Quacchia

Academic Year

2025–2026

Contents

Preface	1
I. Advanced algorithms	2
1. Introduction	3
1.1. Introduction	3
1.1.1. Easy problem - Eulerian circuit	4
1.1.2. Hard problem - Hamiltonian circuit	6
1.2. Complexity of problems	7
1.2.1. Analysis and design of algorithms	7
1.2.2. NP and Computational Intractability	8
1.2.3. NP-completeness	9
2. Approximation Algorithms	12
2.1. Vertex cover	12
2.2. Load balancing	15
2.2.1. List-scheduling algorithm	15
2.3. Center Selection	18
2.4. Metric Traveling Salesman Problem	21
2.4.1. 2-approximation of metric-TSP	21
2.4.2. 1.5-approximation of metric-TSP	23
2.5. The Pricing Method	24
2.5.1. Weighted Vertex Cover	24
3. Local Search	26
3.1. Introduction	26
3.2. Hopfield Neural Networks	27
3.3. Maximum Cut	29
3.4. Genetic algorithms	32
3.4.1. The Maxone problem	32
3.4.2. Travelling Salesman Problem	33
3.5. Nash Equilibrium	34

4. Randomized Algorithms	36
4.1. Introduction	36
4.2. Contention Resolution	37
4.3. Randomized Divide-and-Conquer	38
4.3.1. Quicksort	38
4.4. Numerical algorithms - Buffon's needle	40
 II. Distributed systems	 42
5. Distributed Algorithms	43
5.1. Introduction	43
5.1.1. Distributed environment	43
5.1.2. Complexity measures	45
5.2. Broadcast problem - Flooding	46
5.2.1. Asynchronous execution	48
5.2.2. Correctness of a distributed algorithm	49
5.2.3. N-dimensional mesh	50
5.3. Spanning tree construction	53
5.3.1. Shout protocol	53
5.3.2. Shout+	55
5.4. Leader election	57
5.4.1. Saturation Technique	57
5.4.2. Minimum finding	60
5.5. Ranking problem	62
5.6. Ring topology - Electing the minimum	66
5.6.1. All the way	66
5.6.2. As Far (as it can)	68
5.7. Exercises	70
 6. Synchronous Systems	 71
 7. P2P - Peer to Peer Systems	 75
7.1. Architecture	75
7.2. P2P Network Models	76
7.2.1. Hybrid - Napster	76
7.2.2. Unstructured - Gnutella	76
7.2.3. Hybrid Super Peer - KaZaA	77
7.2.4. Structured - Chord	78
 8. Mobile agents	 81
8.1. Model	81
8.2. Security issues	81
8.2.1. Harmful host - black hole	81

8.2.2. Harmful agent	83
8.3. Robots	87
8.3.1. Gathering problem	88
9. Bitcoin	92
9.1. Bitcoin overview	92
9.1.1. Double-spending problem	93

Preface

Lecture notes for the Advanced and Distributed Algorithms course, based on lectures and slides provided by Prof. *Flaminia Luccio*.

This document has been created and shared exclusively for educational purposes.

The material is available on [GitHub](#) and may be freely distributed for non-commercial use, provided it remains unaltered and the original source is cited.

Part I.

Advanced algorithms

1. Introduction

1.1. Introduction

Graph $G = (V, E)$, V set of vertices, E set of arcs, $n = |V|, m = |E|$

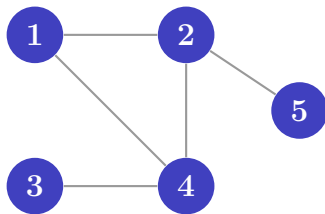


Figure 1.1.: Non-oriented/bidirectional
 $V = \{1, 2, 3, 4, 5\}$
 $E = \{\{1, 2\}, \{1, 4\}, \{2, 4\}, \{2, 5\}, \{3, 4\}\}$

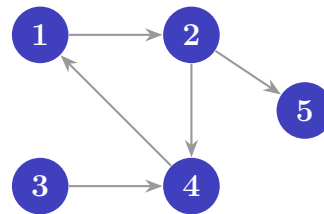


Figure 1.2.: Oriented/Directed
 $V = \{1, 2, 3, 4, 5\}$
 $E = \{(1, 2), (2, 5), (2, 4), (4, 1), (3, 4)\}$

(plain representations of graphs, not the only ones)

A graph can model many real-world problems (streets connecting towns, computers networks, social networks, ...)

Adjacent nodes $u \in V$, its adjacent nodes are $\{v \in V : (u, v) \in E\}$

- **Degree:** num. of indegree and outdegree edges of a vertex in an undirected graph

Path sequence of adjacent nodes (1, 2, 5)

- **Closed path:** path that starts and ends in the same node (1, 2, 4, 1)
- **Simple path:** path without repeated nodes
- **Circuit:** simple closed path

Tree undirected simple graph acyclic and connected (each node is reachable from any other node) ($m = n - 1$)

- **Internal nodes** have degree ≥ 2 , **leaves** have degree 1
- **K-ary tree**: rooted tree in which each node has no more than k children (binary tree)

1.1.1. Easy problem - Eulerian circuit

Eulerian circuit closed path that visits each arc of the graph exactly once.

Königsberg bridge problem Starting from an area, is it possible to come back to the same area by visiting all bridges exactly one time?

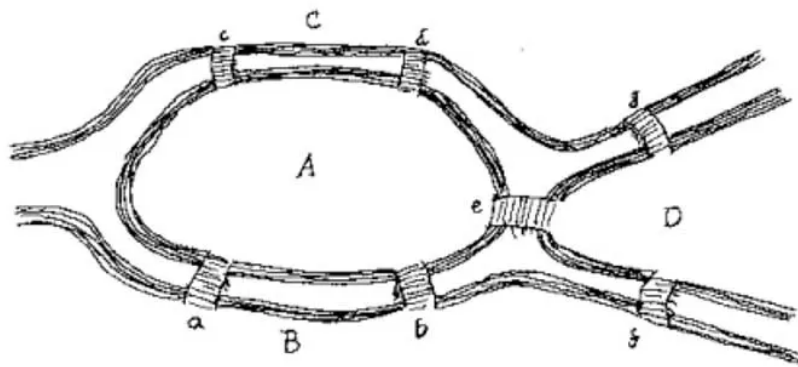


Figure 1.3.: Königsberg bridge problem

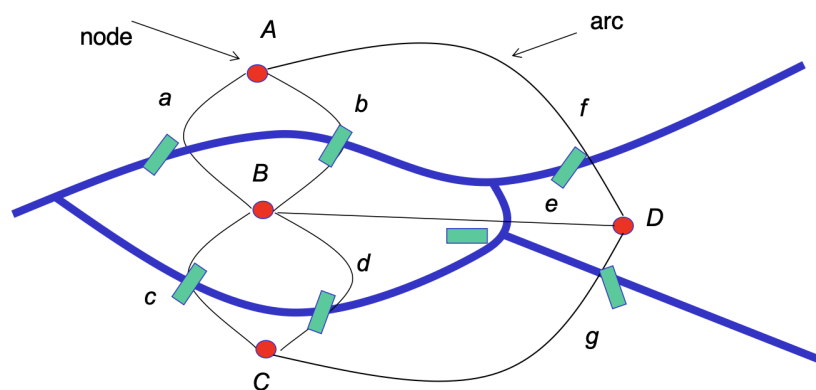


Figure 1.4.: Königsberg graph: a node for each area, an edge for each bridge

Solution: Each vertex must have an even degree to get in and get out. Every odd vertex should be a starting or ending point. In this case, all vertices have an odd degree, so the problem has no solution.

Theorem of Eulero A graph G is Eulerian (it has an Eulerian circuit) $\iff$

1. G is connected
2. All nodes have an even degree

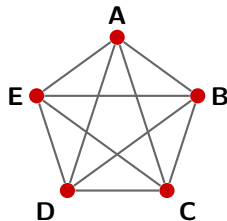
Eulerian circuit algorithm

```

1 #  $O(m) = O(n^2)$ ,  $|E| = m$ ,  $m = n*(n-1)/2$  if  $G$  is complete
2 def Eulero(v1):
3     if v1 has not outgoing arcs: return v1
4     # create a closed path for v1 visiting all arcs exactly once
5     C = [v1, v2, ..., vk, v1]
6     Erase visited arcs in C
7     return (Eulero(v1) , . . . , Eulero(vk), v1)

```

Examples



```

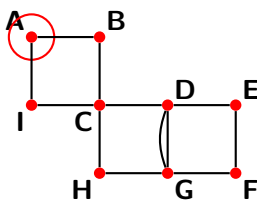
1 Eulero(A):
2     # Find a closed path from A and erase used arcs
3     C = [A,B,C,D,E,A]
4     # Find other closed paths for each vertex
5     # and remove used arcs
6     C[0] = [A,C,E,B,D,A]
7     C = [A,C,E,B,D,A,B,C,D,E,A]

```

```

1 Hamiltonian_circuit = [A, B, C, D, E, A]

```



```

1 Eulero(A):
2     C = [A,B,C,D,E,F,G,H,C,I,A]
3     C[3] = [D,G,D]
4     C = [A,B,C, D,G,D, E,F,G,H,C,I,A]

```

Hamiltonian circuit does not exist because of C

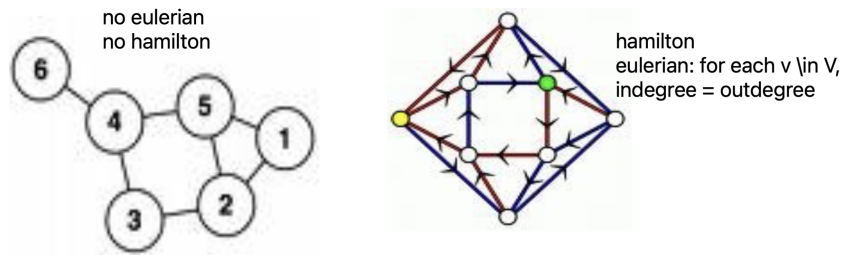


Figure 1.5.: Eulerian/Hamiltonian circuits - other examples

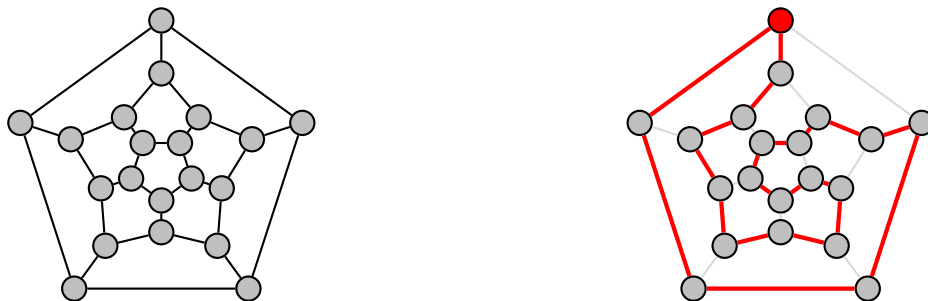
1.1.2. Hard problem - Hamiltonian circuit

Hamiltonian circuit closed path that visits all nodes of the graph exactly once (closed simple path)

Travelling salesman problem given a complete graph with weights on the edges, find the Hamiltonian circuit of minimum weight

The salesman visits each city exactly once and returns to the starting point, travelling the minimal number of kilometers.

Example of a Hamiltonian circuit



Usually, we need to explore all the possible solutions. If the vertexes are labeled, we need to check all the $(n - 1)!$ permutations of the vertexes, starting from a fixed one.

In a complete graph, a Hamiltonian circuit always exists, as we can visit any node from any other node.

In a bipartite graph (two sets of nodes with no edges between nodes of the same set) with an odd number of nodes, a Hamiltonian circuit does not exist.

1.2. Complexity of problems

Easy problem: solvable with an efficient algorithm (polynomial time, $O(n^k)$)

Hard or intractable problem: to find a solution we have to explore all the possible ones (exponential time, $O(k^n)$)

Indecidable problem: no algorithmic solution exists

- **Halting problem:** given an algorithm A and an input, decides if A halts (terminates) on that input (Demonstration)

Polynomial-Time $\exists c > 0, d > 0$ constants such that $T(n) \leq c \cdot n^d$

- **Brute force:** algorithm that checks every possible solution (2^n)

Worst and average case

- **Worst case:** maximum time taken by an algorithm on input of size n
- **Average case:** expected time taken by an algorithm on a random input of size n

Worst-Case Polynomial-Time : An algorithm is efficient if its running time is polynomial

- Some exponential-time algorithms are preferred in practice, because worst-case inputs are rare, or because they are faster on average

1.2.1. Analysis and design of algorithms

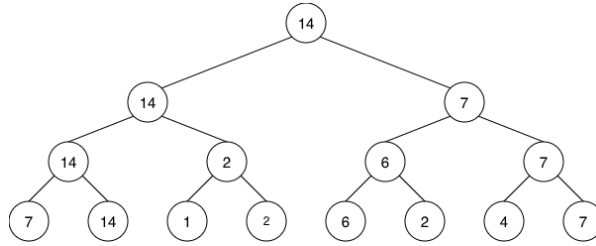
When we devise an algorithm to solve a problem, we need to check correctness (and termination), complexity (worst/avg case with O notation), find a lower bound (Omega notation) and prove optimality (matching upper and lower bounds).

Example - Find the maximum

```
1 def max(V):
2     max_value = V[0]
3     for i in range(1, len(V)):
4         if V[i] > max_value:
5             max_value = V[i]
6     return max_value
```

Complexity: $T(n) = O(n - 1)$, $n-1$ comparisons in worst case.

Lower bound: $\Omega(n - 1)$. As in a tournament, in which the biggest wins, we need at least $n-1$ comparisons ($n-1$ internal nodes in a binary-tree with n leaves).



Optimality: $O(n - 1) = \Omega(n - 1)$, the algorithm is optimal.

1.2.2. NP and Computational Intractability

Decision problem: problem with a yes/no answer. (Given a graph, does an hamiltonian circuit exist?)

Optimization problem: find the best solution that maximizes or minimizes a parameter. (Given a graph, find a shortest path between two nodes)

If the decision problem is NP-complete, then also the optimization problem is NP-complete.

Definitions of P and NP

P Decision problems for which there is a poly-time algorithm to find a solution

- RELPRIME: given two integers, are they coprime? ($\gcd(a, b) = 1$)
- EDITDISTANCE: find edit distance between two strings (dynamic programming)

NP (NP: Non-deterministic poly-time) Decision problems which can be verified in poly-time

- Given a graph and a path, is the path a Hamiltonian circuit?

Given a **certificate** (solution) and a **certifier** (algorithm), we can verify in poly-time if the certificate is a solution to the problem.

- Is p prime? Certificate: d , divisor of p . Certifier: check if d divides p .

- **HAM-CIRCUIT**: Certificate: permutation of nodes. Certifier: check if the permutation is a Hamiltonian circuit.

P $\subseteq$ **NP** Consider $p \in P$, so $\exists A$ algorithm that solves p in poly-time. We can use A as certifier for p , with $t = \varepsilon$, certificate, so $p \in NP$.

EXP Decision problems for which there is an exponential-time algorithm to find a solution ($O(2^{p(n)})$, with $p(n)$ polynomial)

1.2.3. NP-completeness

Polynomial Transformation $P_1 \leq_P P_2$ P_1 is polynomially transformable to P_2 if given any input x to P_1 , we can construct an input y in poly-time, such that x is a yes instance of $P_1 \iff y$ is a yes instance of P_2

P_1 can be reduced to P_2 if any instance of P_1 can be rephrased as an instance of P_2 in poly-time.

Example 1

- P_1 : solution of $ax + b = 0$, P_2 : solution of $a'x^2 + b'x + c' = 0$
- $P_1 \leq_P P_2$: transform $ax + b = 0$ into $0x^2 + ax + b = 0$

Example 2

- P_1 : Does a HAM-CIRCUIT exist in a graph?, P_2 : Does a HAM-CIRCUIT of cost at most k exist in a graph?

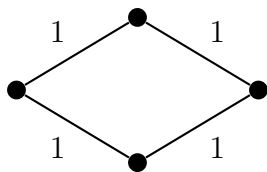


Figure 1.6.: G in P_1
HAM-CIRCUIT exists

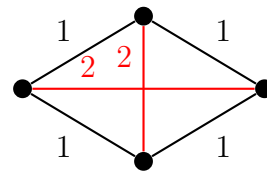


Figure 1.7.: G' in P_2
HAM-CIRCUIT of cost at most $k=4$
exists

NP-Complete class of decisional problems P :

1. $P \in NP$
2. $\forall P' \in NP, P' \leq_P P$

If only the second condition holds, P is **NP-hard**.

Theorem Suppose P_1 is NP-complete. P_1 is solvable in poly-time $\iff P = NP$

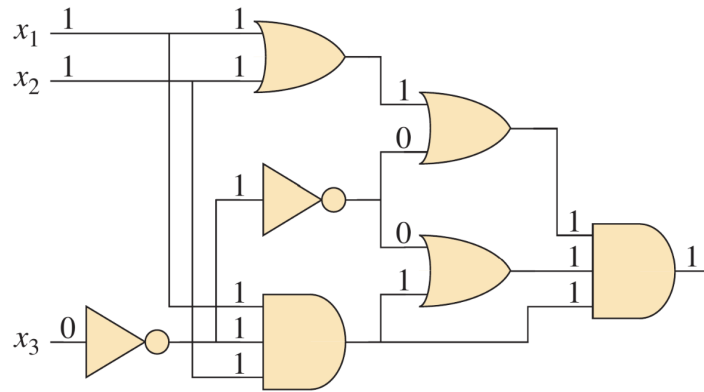
$\Leftarrow$ If $P = NP$, then P_1 is solvable in poly-time, since $P_1 \in NP = P$

$\Rightarrow$ P_1 is solvable in poly-time. Let $X \in NP$, then $X \leq_P P_1$, as P_1 is NP-complete and so X is solvable in poly-time.

$X \in P$. Therefore, $NP \subseteq P$. We already know that $P \subseteq NP$, so $P = NP$.

NP-complete problems

CIRCUIT-SAT Given a boolean circuit, is there an input configuration that makes the output true?



CIRCUIT-SAT is NP-complete [Cook 1971].

In order to show that a problem P_1 is NP-complete, we show that:

- $P_1 \in NP$
- $\exists Q \in NPC : Q \leq_P P_1$

3-SAT Given a boolean formula in conjunctive normal form (CNF), with at most 3 literals per clause, is there an assignment of the variables that makes the formula true?

$$(\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

Certifier: Check that each clause has at least one true literal.

3-SAT is NP-complete, since $3\text{-SAT} \in NP$ and $\text{CIRCUIT-SAT} \leq_P 3\text{-SAT}$.

Clique Given an undirected graph $G = (V, E)$, a clique is $V' \subseteq V$ such that $\forall u, v \in V', (u, v) \in E$.

(Complete subgraph of G)



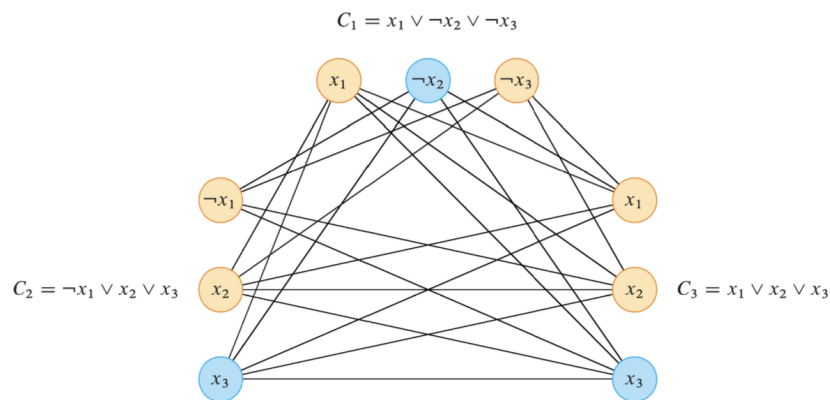
A complete graph with $|V| = n$ has all cliques from size 1 to n .

Optimization clique problem: Find a clique of maximum size in a graph.

Decision clique problem (DCLIQUE): Given a graph G and an integer k , determine whether G has a clique of size k .

DCLIQUE is NP-complete

1. $DCLIQUE \in NP$: given a graph G , an integer k and a certificate $V' \subseteq V$, we can check in poly-time ($O(|V|^2)$) that $\forall u, v \in V', u \neq v, (u, v) \in E$
2. $3\text{-SAT} \leq_P DCLIQUE$: given a 3-SAT formula with k clauses, we construct a graph with a vertex for each literal and edges between non-contradicting literals from different clauses.



The graph has a k -clique $\iff$ the formula is satisfiable

2. Approximation Algorithms

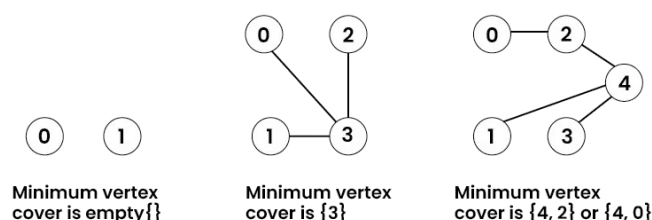
Efficient algorithms that find approximate solutions to optimization problems (NP-hard), ensuring a close distance to optimal solutions.

ρ -approximation algorithm Algorithm that guarantees to solve arbitrary instances of the problem and to find a solution in poly-time within ratio ρ of true optimum

Assume C^* : cost of an optimal solution, C : cost of approximation algorithm, then we have a ρ -approximated algorithm $\iff \max(C/C^*, C^*/C) \leq \rho$

2.1. Vertex cover

Given a graph $G = (V, E)$ and an integer k , is there $V' \subseteq V$, such that $|V'| \leq k$ and $\forall (u, v) \in E \rightarrow u \in V' \vee v \in V'$?



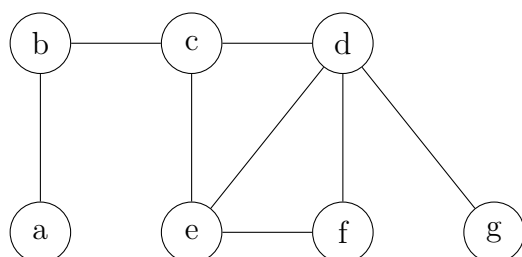
- Optimization: Find vertex cover of minimum size
- Decision: Does G have vertex cover of size $\leq k$? (NP-complete, $3\text{-SAT} \leq_p \text{VC}$)

```
1 def vertex_cover_2_approximation(V, E):
2     VC, EC = {}, E
3     while EC:
4         Extract an arbitrary edge (u,v) from EC
5         VC = VC U {u,v} # add endpoints to VC
6         Erase incident edges of u,v from EC
7     return VC
```

Complexity $O(n \cdot m)$, at most m iterations, in which removing incident edges costs at most $O(n)$

($O(n + m)$ with adjacency list as we process each edge and each vertex at most once)

Example 1



$$VC^* = \{e, d, b\}$$

$$VC = \emptyset$$

$$EC = E$$

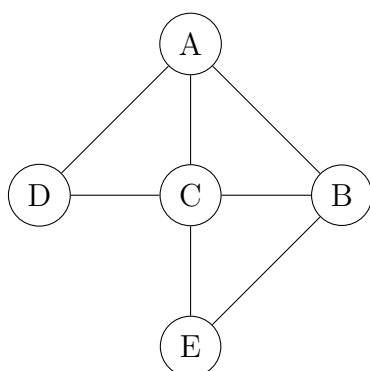
$$VC = \{e, d\}$$

$$EC = \{\{a, b\}, \{b, c\}\}$$

$$VC = \{e, d, b, c\}$$

$$EC = \emptyset$$

Example 2



$$VC^* = \{A, B, C\}$$

$$VC = \emptyset$$

$$EC = E$$

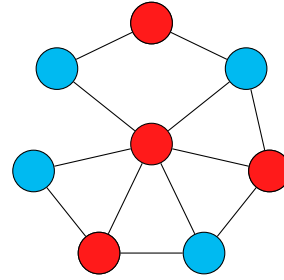
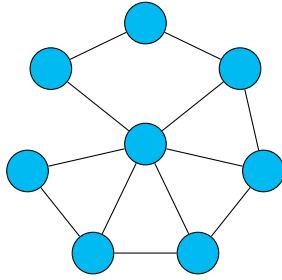
$$VC = \{A, B\}$$

$$EC = \{\{C, D\}, \{C, E\}\}$$

$$VC = \{A, B, C, D\}$$

$$EC = \emptyset$$

Example 3



The best solution must contain at least one of the endpoints for each edge.

Correctness VC is a vertex cover and the algorithm terminates, as at each step one edge is removed.

Approximation Let VC^* be the optimal solution and A the set of extracted edges $\{(a, b), (c, d), (e, f)\}$

1. VC^* must contain at least one node for each edge in A , and all the edges in A are disjoint, as when we extract an edge, we remove all the edges incident to its endpoints $|VC^*| \geq |A|$

2. $|VC| = 2 \cdot |A|$

So $|VC| = 2|A| \leq 2|VC^*|$

$(C/C^* = |VC|/|VC^*| = 2 = \rho)$

2.2. Load balancing

Given m machines and n jobs with processing times $t_1, t_2, \dots, t_n$

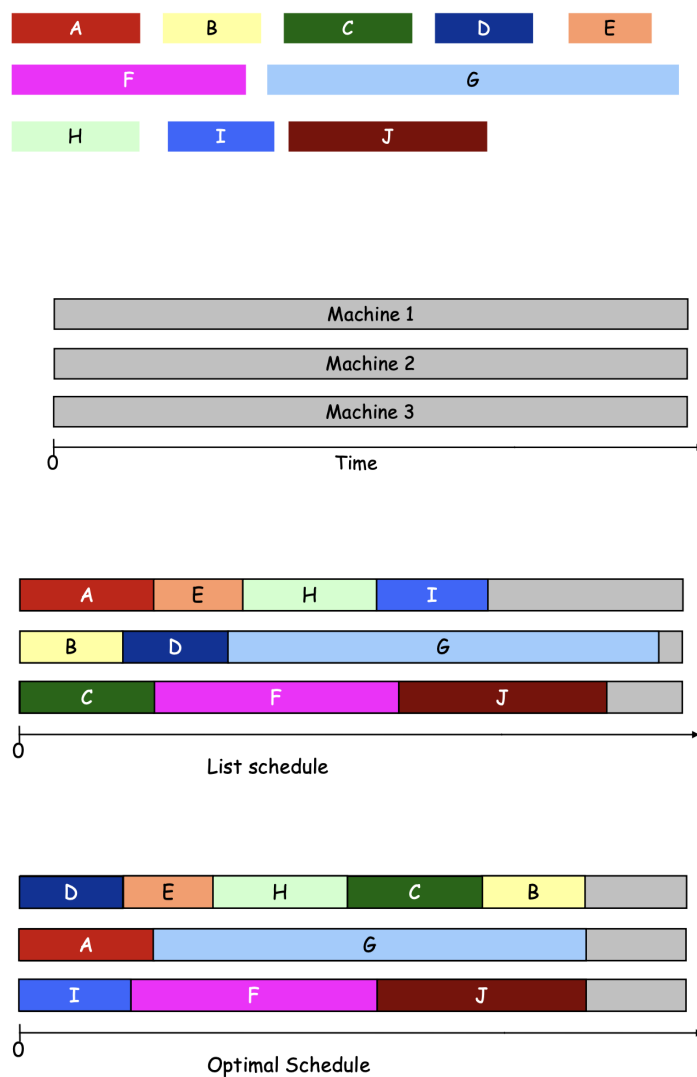
Load on a machine i : $L_i = \sum_{j \in J_i} t_j$

Makespan is the maximum load on any machine: $L = \max_i L_i$

Assign each job to a machine to minimize makespan (NP-hard)

2.2.1. List-scheduling algorithm

Assign the job to the machine with the smallest current load



```

1 # m: machines, n: jobs, T: processing times
2 def list_scheduling(m, n, T):
3
4     L: array of machine loads of size m, init to 0
5     J: array of assigned jobs of size m, init to {}
6
7     for j in range(n):
8         i = argmin(L)           # machine with current min load
9         J[i] = J[i] ∪ {j}       # assign job j to machine i
10        L[i] = L[i] + T[j]      # update load of machine i
11
12    return J

```

Complexity: $O(n \log m)$ with a priority queue (insert and extract-min $O(\log m)$)

Theorem. It is a 2-approximation greedy algorithm, as it makes the locally optimal choice at each stage [Graham, 1966]

Lemma 1. $L^* \geq \max_j t_j$, a machine must process the most time-consuming job

Lemma 2. $L^* \geq \frac{1}{m} \sum_j t_j$, a machine must process at least the average load

Proof. Let i be the machine with the smallest load before assigning job j

$$L_i - t_j \leq L_k \quad \text{for all } 1 \leq k \leq m$$

Sum over all k and divide by m :

$$\begin{aligned}
 m(L_i - t_j) &\leq L_1 + L_2 + \cdots + L_m = \sum_K L_k \\
 L_i - t_j &\leq \frac{1}{m} \sum_K L_k \\
 &= \frac{1}{m} \sum_K t_k && \text{(by def. of Load)} \\
 &\leq L^* && \text{(by lemma 2)}
 \end{aligned}$$

So $L_i - t_j \leq L^*$ and $t_j \leq L^*$ (by lemma 1) $\rightarrow L_i = (L_i - t_j) + t_j \leq 2L^*$ □

Longest processing time (LPT)

```

1 def LPT_list_scheduling(m, n, T):
2     # sort jobs in descending order of processing time
3     sorted_jobs = sorted(T, reverse=True)
4     return list_scheduling(m, n, sorted_jobs)

```

N.B. If $n \leq m \rightarrow \text{list_scheduling}$ is optimal, as each job is assigned to a different machine.

Lemma 3. If $n > m \rightarrow L^* \geq 2t_{m+1}$

- $n > m$, so at least one machine gets two jobs
- t_i are in descending order, so each requires at least t_{m+1}

$L_i - t_j \leq L^*$ and $t_j \leq \frac{1}{2}L^*$ (by lemma 3) $\rightarrow L_i \leq \frac{3}{2}L^*$

Considering the case $n > m$, then $j \geq m + 1 \rightarrow t_j \leq t_{m+1} \leq \frac{1}{2}L^*$

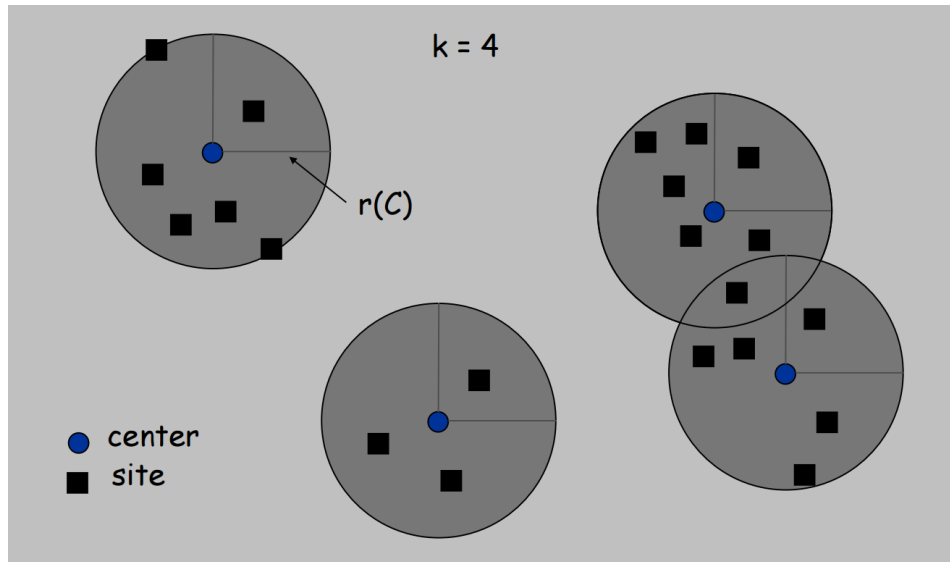
In reality, LPT is a $4/3$ -approximation algorithm.

Example $m = 4$ machines, $T = [1, 3, 2, 6, 1, 2]$

1	# List scheduling	# LPT: $T = [6, 3, 2, 2, 1, 1]$
2	1. 1 1 2	1. 6
3	2. 3	2. 3
4	3. 2	3. 2 1
5	4. 6	4. 2 1

2.3. Center Selection

Given a set of n sites $s_1, \dots, s_n$ and integer $k > 0$, place k centers C to minimize the maximum distance from any site to its closest center.



- $\text{dist}(x, y)$: distance x - y
- $\text{dist}(s_i, C) = \min_{c \in C} \text{dist}(s_i, c)$: distance from site s_i to closest center
- $r(C) = \max_i \text{dist}(s_i, C)$: smallest covering radius

Properties

$\text{dist}(x, x) = 0$ (identity)

$\text{dist}(x, y) = \text{dist}(y, x)$ (symmetry)

$\text{dist}(x, y) \leq \text{dist}(x, z) + \text{dist}(z, y)$ (triangle inequality)

```
1 def greedy_center_selection(k, n, S):
2     if k >= n: return S      # place a center at each site
3     C = { s }                # init centers with an arbitrary site
4     for _ in range(k-1):
5         # add site s farthest from any center with max dist(s,C)
6         C = C | { farthest_site(S,C) }
7     return C
```


Example $n = 4, k = 2$

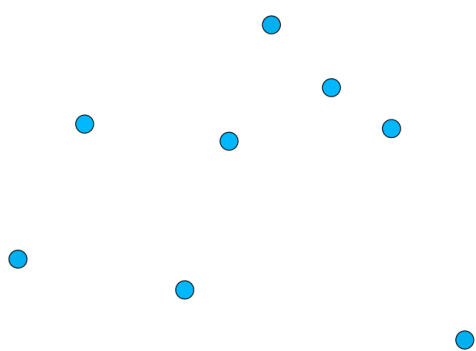


Figure 2.1.: Step 1

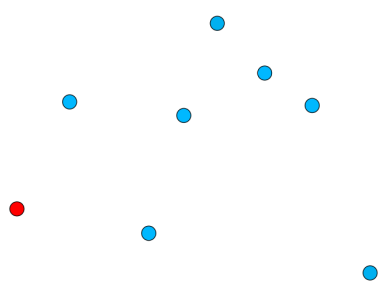


Figure 2.2.: Step 2

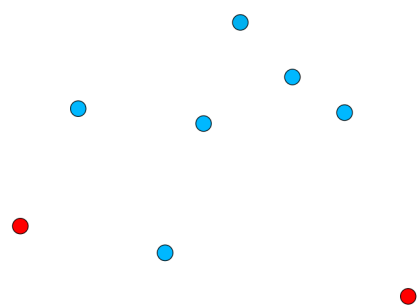


Figure 2.3.: Step 3

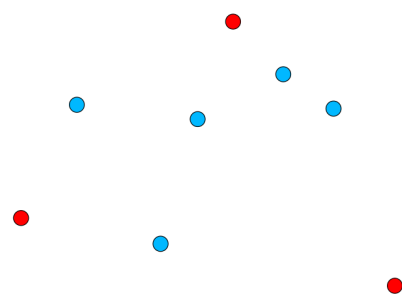


Figure 2.4.: Step 4

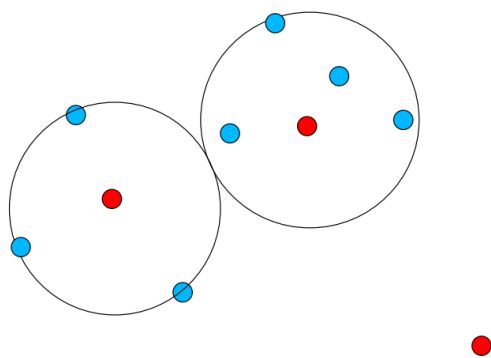


Figure 2.5.: Optimal solution $r(C^*)$

Theorem. The greedy center selection algorithm is a 2-approximation algorithm. Then $r(C) \leq 2r(C^*)$

Proof.(by contradiction) Assume $r(C) > 2r(C^*) \iff r(C^*) < \frac{1}{2}r(C)$

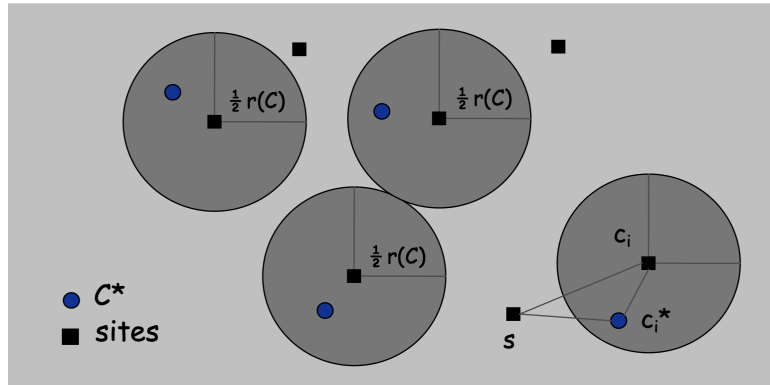
Let C be the approximate solution and C^* the optimal one.

- Consider a ball of radius $\frac{1}{2}r(C) \quad \forall c_i \in C$
- Exactly one $c_i^* \in C^*$ covers exactly one center $c_i \in C$, because $r(C^*) < \frac{1}{2}r(C)$

Consider any $s \in S$

$$\begin{aligned}
 \text{dist}(s, C) &\leq \text{dist}(s, c_i) && \text{(definition of dist(s, C))} \\
 &\leq \text{dist}(s, c_i^*) + \text{dist}(c_i^*, c_i) && \text{(by triangle inequality)} \\
 &\leq 2r(C^*) && (c_i^* \text{ covers both } s \text{ and } c_i)
 \end{aligned}$$

Thus $r(C) \leq 2r(C^*)$, a contradiction.



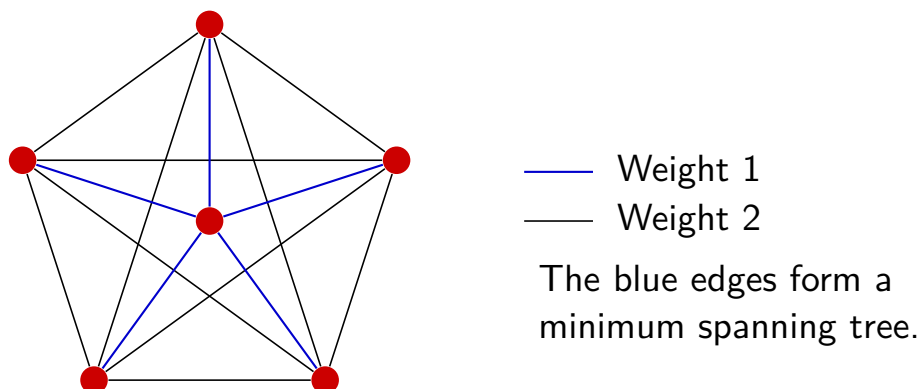
□

This is the best possible approximation ratio.

2.4. Metric Traveling Salesman Problem

We consider a special case of TSP: a complete graph with non-negative edge costs satisfying triangle inequalities

For any three vertices $u, v, x : w(u, v) \leq w(u, x) + w(x, v)$



2.4.1. 2-approximation of metric-TSP

1. Find the minimum spanning tree $MST(G)$

Spanning tree Given a connected, undirected graph, a spanning tree is a subgraph that is a tree and connects all the vertices together.

Minimum spanning tree (MST) Spanning tree with the smallest total edge weight, than any other spanning tree.

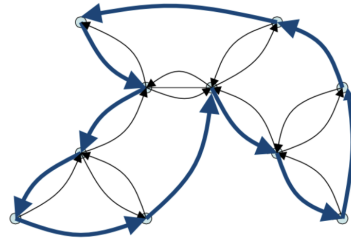
Prim's algorithm finds a MST in $O(m \log n)$.

2. Double the edges of $MST(G)$ to get an Eulerian graph (connected and all vertices have even degree)



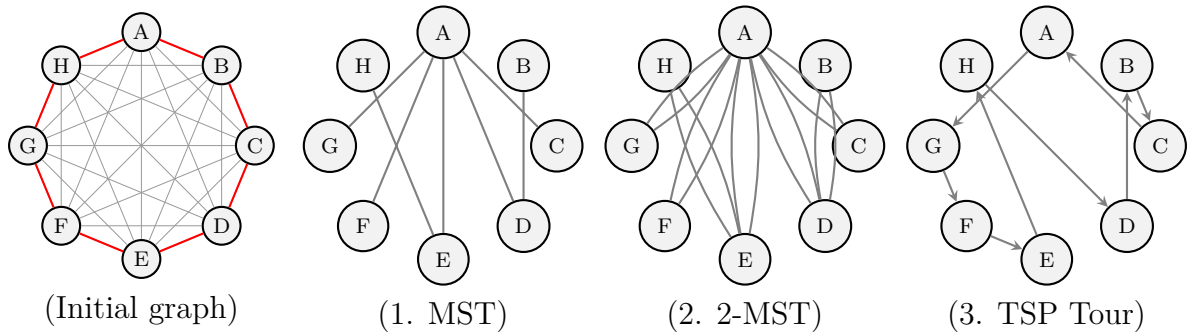
We can traverse the Eulerian graph, visiting each edge exactly once.

3. Traverse the Eulerian graph and use shortcuts to skip already visited vertices, to create a Hamiltonian tour



The initial graph is complete and satisfies the triangle inequality, so the shortcut does not increase the tour cost.

Example



Theorem. It is a 2-approximation greedy algorithm

Proof. Let T be the MST found in step 1, OPT the optimal TSP tour (set of edges), A set of edges found by the algorithm and EC set of edges in the Eulerian cycle.

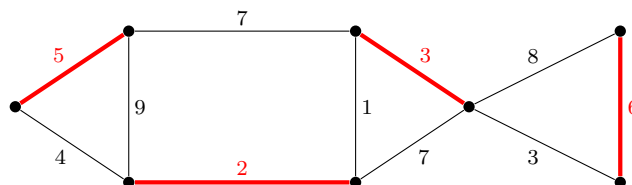
$$\begin{aligned}
 \text{cost}(T) &\leq \text{cost}(OPT) && \text{(T is a MST, OPT is a cycle)} \\
 \text{cost}(EC) &= 2 \cdot \text{cost}(T) && \text{(by step 2)} \\
 \text{cost}(A) &\leq \text{cost}(EC) && \text{(by step 3, triangle inequality)}
 \end{aligned}$$

Thus, $\text{cost}(A) \leq \text{cost}(EC) = 2\text{cost}(T) \leq 2\text{cost}(OPT)$ □

2.4.2. 1.5-approximation of metric-TSP

Matching M subset of edges such that no two edges share a common vertex

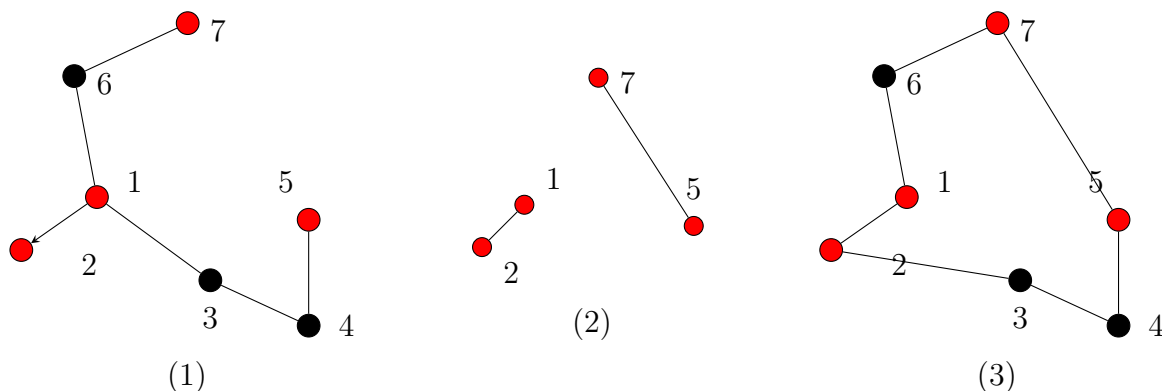
- **Weight** of M: sum of its edge weights
- **Perfect matching**: matching that covers every vertex of the graph



It is findable in poly-time

1.5 approximation algorithm

1. Find MST
2. Find M: minimum weight perfect matching on odd degree nodes of MST
3. Find TSP tour of $EC = MST + M$



(Any graph has a even number of odd degree nodes $\implies$ M exists)

Proof.

$$\text{cost}(T) \leq \text{cost}(OPT) \quad (T \text{ is MST, } OPT \text{ is a cycle})$$

$$\text{cost}(M) \leq 0.5 \cdot \text{cost}(OPT) \quad (OPT \text{ is a cycle with 2 matchings, } M \text{ is the minimum})$$

$$\text{cost}(EC) = \text{cost}(T) + \text{cost}(M) \leq 1.5 \cdot \text{cost}(OPT)$$

$$\text{cost}(A) \leq \text{cost}(EC) \leq 1.5 \cdot \text{cost}(OPT)$$

□

2.5. The Pricing Method

Approximation-preserving reduction Algorithm that transforms one optimization problem into another problem, preserving the approximability of the original problem.

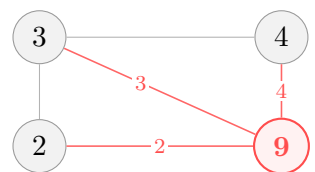
2.5.1. Weighted Vertex Cover

Given a graph with vertex weights, find a vertex cover of minimum weight.

Each edge $e = (i, j)$ pays price $p_e \geq 0$ to use vertex i and j

Fairness Edges incident to vertex i pay at most w_i (vertex weight)

$$\forall \text{ vertex } i : \sum_{e=(i,j)} p_e \leq w_i$$



$$4 + 3 + 2 = 9 \leq 9$$

Tight node $\sum_{e=(i,j)} p_e = w_i$

Lemma. For any vertex cover VC and any fair prices p_e , $\sum_e p_e \leq w(VC)$

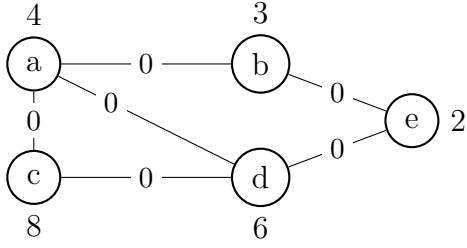
Proof.

$$\begin{aligned} \sum_e p_e &\leq \sum_{i \in VC} \sum_{e=(i,j)} p_e && \text{(VC is a vertex cover)} \\ &\leq \sum_{i \in VC} w_i = w(VC) && \text{(by definition of fairness)} \end{aligned}$$

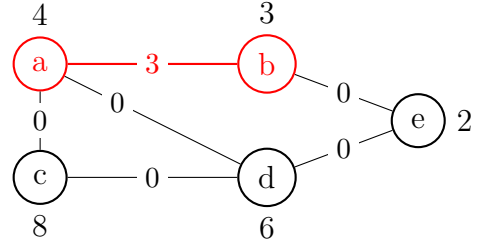
□

```
1 def weighted_vertex_cover_approx(G, w):
2
3     VC, EC = {}, E # edges init to 0
4
5     while exists an edge that can be increased to make a node tight:
6         Increase that edge as much as possible
7         Add new tight node to VC
8
9     return VC
```

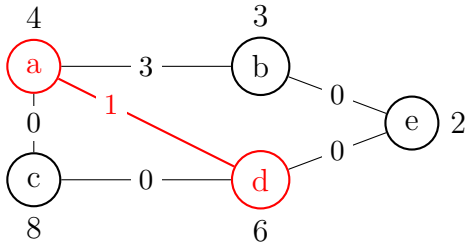
Example



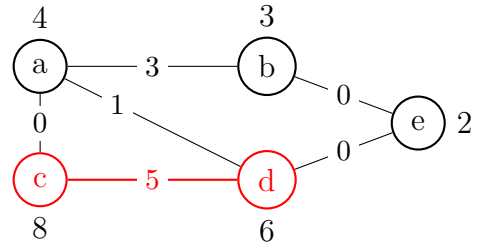
Init prices



Extract a-b, b is tight



Extract a-d, a is tight



Extract c-d, d is tight

$$VC = \{a, b, d\}, \quad w(VC) = 13$$

$$VC^* = \{a, d, e\}, \quad w(VC^*) = 12$$

Theorem. Pricing method is a 2-approximation

Proof.

- Termination: At each step, at least one node becomes tight and edges are finite.
- Correctness: All edges are covered, since if an edge is uncovered, then its endpoints are not tight and the loop would not terminate.
- Approximation: Let VC^* be the optimal vertex cover, then $w(VC) \leq 2w(VC^*)$

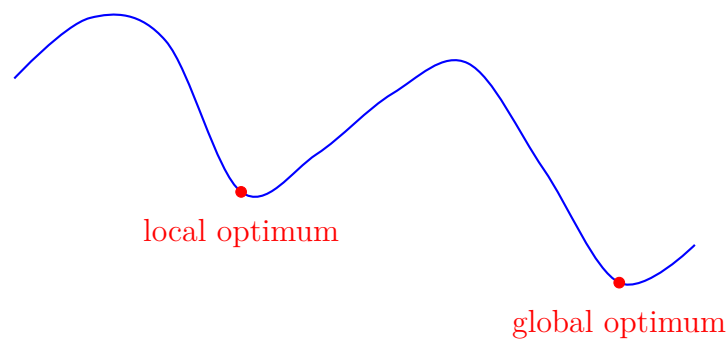
$$\begin{aligned}
 w(VC) &= \sum_{i \in VC} w_i = \sum_{i \in VC} \sum_{e=(i,j)} p_e && \text{(nodes in VC are tight)} \\
 &\leq \sum_{i \in V} \sum_{e=(i,j)} p_e && (VC \subseteq V, \text{ prices } \geq 0) \\
 &= 2 \sum_e p_e && \text{(each edge is counted twice)} \\
 &\leq 2w(VC^*) && \text{(by fairness lemma)}
 \end{aligned}$$

□

3. Local Search

3.1. Introduction

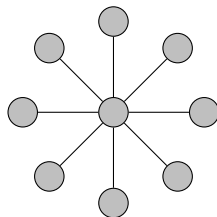
It is a heuristic for optimization problems that starts with an initial solution and moves to better ones in its neighborhood until it reaches a local optimum.



Gradient Descent: Vertex Cover The initial solution is the set of all vertices or an empty set. At each step, we add or remove a vertex ensuring that all edges are covered.

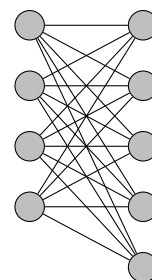
Examples

Star graph

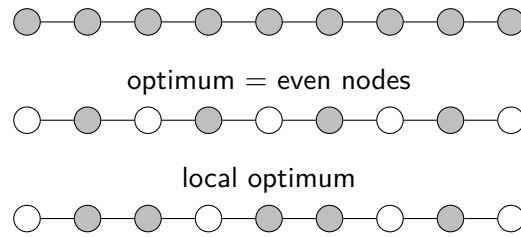


- Optimum: only center node
- Local optimum: all other nodes

Bipartite graph



- Optimum: all left-side nodes
- Local optimum: all right-side nodes



Metropolis algorithm Probabilistic method that allows moves to worse solutions with a certain probability to escape local minima.

Gibbs-Boltzmann function $P(\text{system in a state with energy } E)$ is proportional to $e^{-E/kT}$
 $T > 0$ (temperature), $k > 0$ (constant)

Other algorithms exist in order to escape from plateaus: *Simulated Annealing*, *Mixed random walk*.

3.2. Hopfield Neural Networks

Simple model of an associative memory, in which neurons are connected to each other and can be in one of two states. The network evolves in order to reach a stable state.

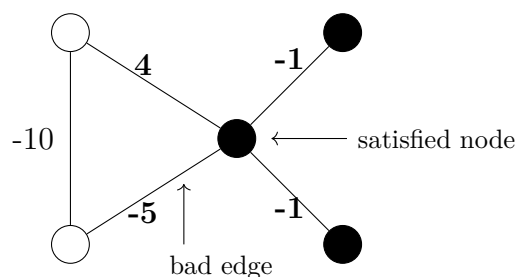
Given an undirected weighted graph with nodes in states $\{\pm 1\}$

- if $w_{uv} < 0$ then u and v want to have the same state
- if $w_{uv} > 0$ then u and v want different states

In a configuration S

- Edge $e = (u, v)$ is **good** if $w_{uv}s_us_v < 0$
- A node u is **satisfied** if $\sum_v w_{uv}s_us_v \leq 0$ ($\sum |w_{\text{good}}| \geq \sum |w_{\text{bad}}|$)

A configuration is **stable** if all nodes are satisfied.



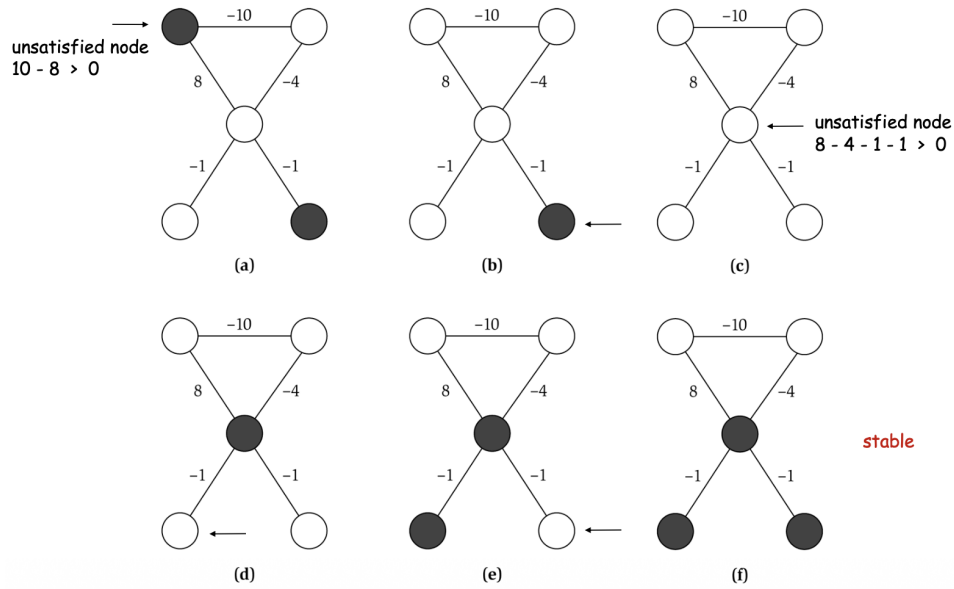
State-flipping algorithm

```

1 def hopfield_flip(G, w):
2     S: arbitrary configuration
3     while S is not stable: flip_any_unsatisfied_node(S)
4     return S

```

Example



Claim. It terminates with a stable configuration in at most $W = \sum_e |w_e|$ iterations

Proof. Let $\phi(S) = \sum |w_{\text{good}}| \implies 0 \leq \phi(S) \leq W$ (weights are integers)

When we flip the state of node u , all the edges incident to u change their state

Let g_u, b_u be the total absolute weight on good and bad edges incident to u

$$\phi(S') = \phi(S) - g_u + b_u$$

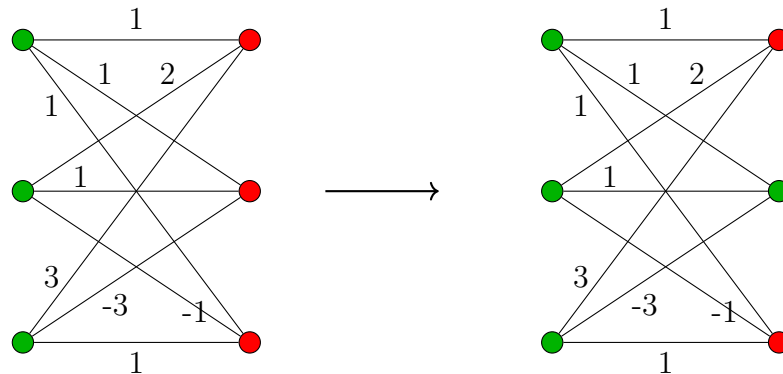
Since u was unsatisfied, $b_u > g_u \iff b_u \geq g_u + 1 \implies \phi(S') \geq \phi(S) + 1$

Thus, the absolute value of good edges increases at least by 1 at each iteration and the max value can be W , so the algorithm terminates in at most W iterations.

□

Complexity: It depends on weights. If they are high, the algorithm becomes infeasible.

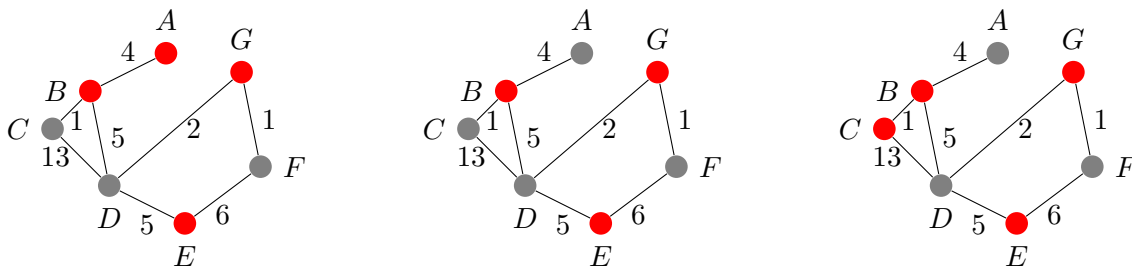
Example 1: Only the second red node is unsatisfied, flip its state and the configuration becomes stable.



Example 2

AB, CD, are bad edges. A, C, D are unsatisfied

- flip A $\implies$ AB is good, A is stable; CD is still bad
- flip C $\implies$ CD is good, C is stable; CB is bad



When all weights are positive, it is equivalent to the Max Cut problem.

3.3. Maximum Cut

Given an undirected graph, with positive integer edge weights, find a node partition (A, B) that maximizes the total weight of edges crossing the cut.

$$W(A, B) = \sum_{u \in A, v \in B} w_{uv}$$

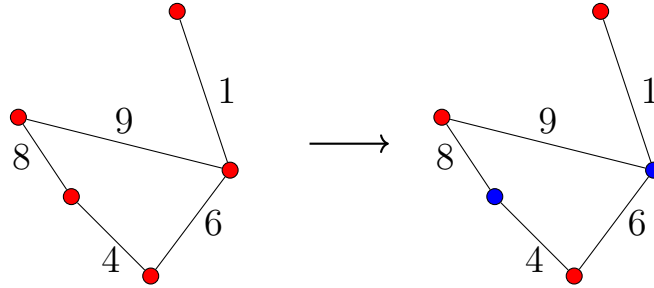
A node v is **satisfied** if the sum of edge weights to the other partition is $\geq$ the sum of edge weights to the same partition.

```

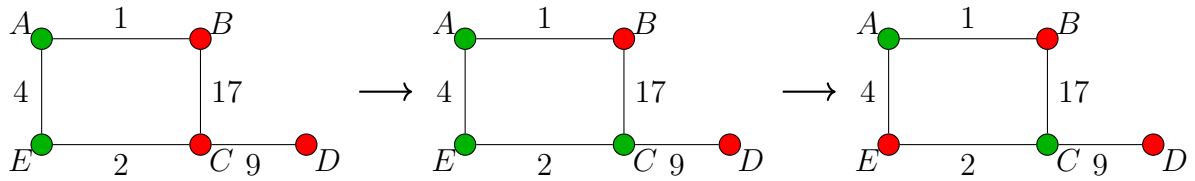
1 # greedy algorithm
2 def max_cut_local(G, w):
3     (A, B) = random node partition
4
5     while exists unsatisfied node v: move v to the other partition
6
7     return (A, B)

```

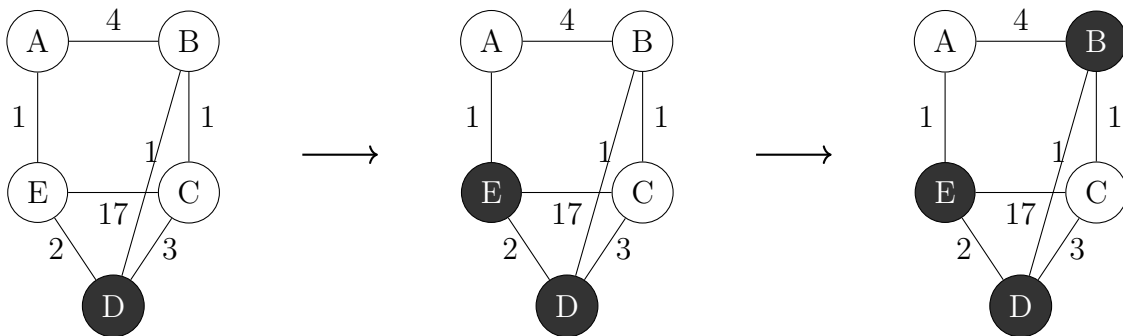
Example 1



Example 2



Example 3



Theorem. Let (A, B) be a local optimum and (A^*, B^*) be a global optimum. Then,

$$w(A, B) \geq \frac{1}{2} \sum_e w_e \geq \frac{1}{2} W(A^*, B^*)$$

.

Proof.

$$\forall u \in A : \sum_{v \in A} w_{uv} \leq \sum_{v \in B} w_{uv} \quad (\text{by local optimality})$$

$$2 \sum_{\{u,v\} \subseteq A} w_{uv} \leq \sum_{u \in A, v \in B} w_{uv} = w(A, B) \quad (\text{sum over all } u \text{ in } A)$$

$$2 \sum_{\{u,v\} \subseteq B} w_{uv} \leq \sum_{u \in A, v \in B} w_{uv} = w(A, B) \quad (\text{sum over all } u \text{ in } B)$$

$$\sum_{\{u,v\} \subseteq A} w_{uv} + \sum_{\{u,v\} \subseteq B} w_{uv} \leq w(A, B) \quad (\text{sum the two inequalities and divide by 2})$$

$$W = \sum_{\{u,v\} \subseteq A} w_{uv} + \sum_{\{u,v\} \subseteq B} w_{uv} + w(A, B) \leq 2w(A, B) \quad (\text{considering all edges})$$

$$\text{Considering } w(A^*, B^*) \leq W \implies w(A, B) \geq \frac{1}{2}W \geq \frac{1}{2}w(A^*, B^*) \quad \square$$

N.B. Better approximation algorithms exist for this problem. We can also flip more than one node at each step.

3.4. Genetic algorithms

Class of optimization algorithms inspired by the process of natural selection (Darwin 1859). Start with a population of candidate solutions that evolve by iteratively applying a set of stochastic operators.

Population It can be a set of binary strings, real numbers, permutations, rules, ...

Evaluation The evaluator assigns a fitness score to each candidate solution based on how well it solves the problem.

Stochastic operators

- **Selection:** Replicate most successful candidates based on their fitness score
- **Recombination (cross-over):** Combine parts of two solutions to create a new one
- **Mutation:** Randomly perturbs a candidate solution

In the Generational GA the entire population is replaced at each iteration, while in the Steady-state GA only a few candidates are replaced at each iteration.

GA are widely used in optimization problems, machine learning, robotics, ...

3.4.1. The Maxone problem

We want to maximize the number of ones in a string of l bits. The fitness function f is the number of ones in the string and the population is a set of n binary strings.

Example

$$\begin{array}{lll} s_1 = 1111010101 & f(s_1) = 7 & s_3 = 1110110101 & f(s_3) = 7 & s_5 = 1110111101 & f(s_5) = 8 \\ s_2 = 0111000101 & f(s_2) = 5 & s_4 = 0100010011 & f(s_4) = 4 & s_6 = 0100110000 & f(s_6) = 3 \end{array}$$

Selection: We can use **roulette wheel selection**, where the probability of selecting a candidate is proportional to its fitness score $\frac{f(i)}{\sum_i f(i)}$.

Crossover: We randomly decide whether to perform crossover. If we do, we randomly select a crossover point for each couple and create two new candidates combining the parts of the parents.

$$\begin{array}{ll} s_1 = 11\textcolor{red}{1010101} \rightarrow 11\textcolor{red}{1000101} & s_5 = 11101\textcolor{red}{11101} \rightarrow 11101\textcolor{red}{10000} \\ s_2 = 0\textcolor{green}{111000101} \rightarrow 0\textcolor{green}{111010101} & s_6 = 0\textcolor{green}{100110000} \rightarrow 0\textcolor{green}{100111101} \end{array}$$

Mutation: Small probability of flipping each bit.

$$\begin{array}{llll} s_1 = 11\textcolor{red}{0}1000101 & f(s_1) = 5 & s_3 = 1110110101 & f(s_3) = 7 \\ s_2 = 0111\textcolor{red}{1}10101 & f(s_2) = 7 & s_4 = 010\textcolor{red}{1}010011 & f(s_4) = 5 \\ s_5 = 1110110000 & f(s_5) = 5 & s_6 = 010\textcolor{red}{1}11\textcolor{red}{0}101 & f(s_6) = 6 \end{array}$$

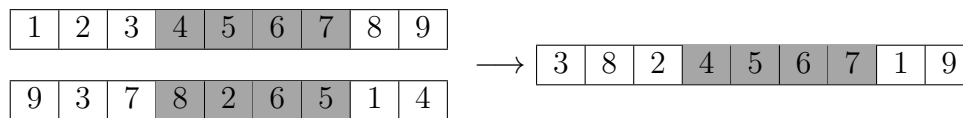
In a generation, the total population fitness changed from 34 to 35, with a 2.9% improvement. We continue iterating until a stopping criterion is met.

3.4.2. Travelling Salesman Problem

Find a Hamiltonian circuit of minimum cost. If cities are labeled from 1 to n , a solution is a permutation of $\{1, \dots, n\}$.

Selection: The fitness function is the inverse cost of the tour, $f = 1/t$ (GA are maximization algorithms).

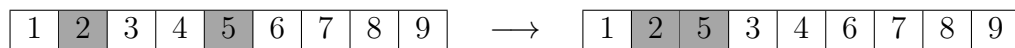
Crossover: custom approach in which we randomly copy a subset of cities from a parent and fill the remaining cities in the order they appear in the other parent.



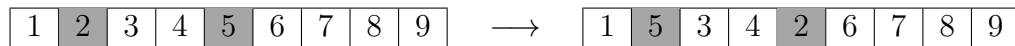
First, copy (4, 5, 6, 7) from the first parent, then copy the rest (1, 2, 3, 8, 9) according to the order in the second parent (1, 9, 3, 8, 2) starting from cut point. Repeat for the second child.

Mutations

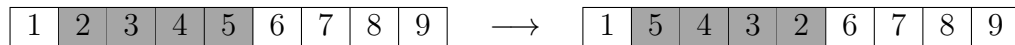
- Insert mutation: randomly choose two cities, move the second close to the first one and shift the others



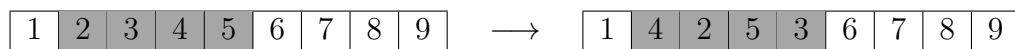
- Swap mutation: randomly choose two cities and swap them



- Inversion mutation: randomly choose two cities and reverse the order of the cities between them



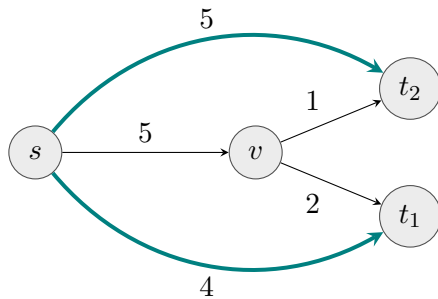
- Scramble mutation: randomly choose a subset of cities and randomly shuffle them



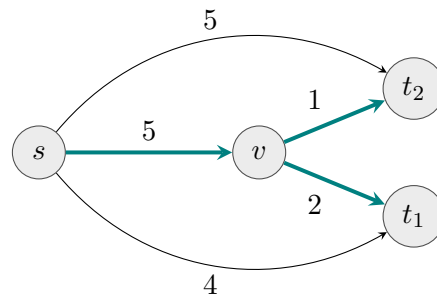
3.5. Nash Equilibrium

Multicast routing Given a directed graph with edge costs $c_e \geq 0$, a source node s , k agents starting from s and terminating in $t_1, \dots, t_k$ different destinations, each agent j must choose a path from s to t_j .

Fair share if n agents use the same edge, each of them pays c_e/n .



$$A_1 = 4, \quad A_2 = 5$$



$$A_1 = 5/2 + 1, \quad A_2 = 5/2 + 2$$

Agents improve their solution in response to changes made by other agents (best response dynamics), until reaching a stable solution called **Nash equilibrium**, where no agent has an incentive to switch.

```

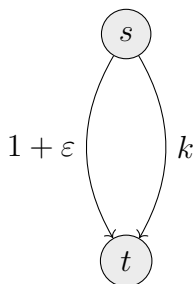
1 best_response_dynamics(G, c):
2     pick a path for each agent
3
4     while not in Nash equilibrium:
5         pick an agent who can improve by switching paths and switch its path

```

Drawback: In a Nash equilibrium, each agent acts individually.

In the example above, if both agents start in the middle, then one switches to the external one, followed by the other, they reach a Nash equilibrium, but it is globally worse than the initial situation.

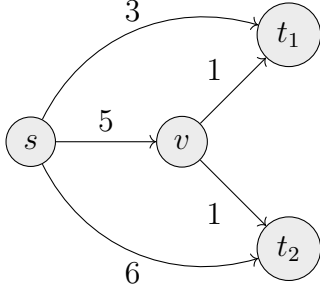
Social optimum Minimize the total global cost to all agents.



If all agents choose the left path, each agent pays $(1 + \varepsilon)/k$ and the total cost is $1 + \varepsilon$.

If all agents choose the right path, each agent pays $k/k = 1$ and the total cost is k .

Example 1: Find social optimum and Nash equilibrium when both agents initially choose the central path.



$$A_1 : [s - v - t_1] \quad (3.5)$$

$$A_2 : [s - v - t_2] \quad (3.5)$$

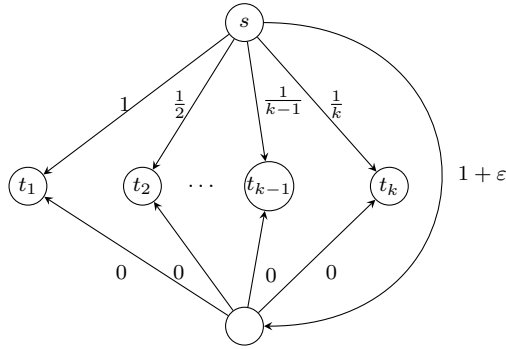
social optimum: 7

$$A_1 : [s - t_1] \quad (3)$$

$$A_2 : [s - v - t_2] \quad (6)$$

Nash equilibrium: 9

Example 2



If all agents choose the right path, each pays $(1 + \varepsilon)/k$, with total cost $1 + \varepsilon$ (social optimum).

Then, last one would switch to left path, paying $1/k < (1 + \varepsilon)/k$, reducing the individual cost to $(1 + \varepsilon)/(k - 1)$

Then, the previous one would switch too, paying $1/(k - 1) < (1 + \varepsilon)/(k - 1)$ and followed by the others, until all agents switch to the left path, with total cost $\ln(k) + \gamma$.

Note: In standard local search, the global minimum is always a stable solution. Here, the social optimum can be an unstable solution.

Theorem (Price of stability). The total cost of the best Nash equilibrium exceeds the social optimum by at most a factor of $H(k) = 1 + 1/2 + \dots 1/k$ (harmonic number)

Existence: Nash equilibrium always exists in this problem, but it can not be computed in poly-time

4. Randomized Algorithms

4.1. Introduction

Algorithms that make random choices during the execution. They may provide wrong result with a small probability, but they are average faster than deterministic algorithms.

We use pseudo-random generators, in which a seed produce the same random sequence. Choices should have a uniform distribution over $[0, 1)$.

Numerical algorithms approximate solutions. $[2 \pm 0.5, 2.3 \pm 0.5, 2.1 \pm 0.5]$

Monte Carlo algorithms correct answer with high probability. $[2 - 2 - 1 - 2 - 2]$

- Contraction algorithm for global min cut

Las Vegas algorithms always correct answer, may not terminate. $[2 - \text{no output} - 2]$

- Randomized quicksort, Johnson's MAX-3SAT algorithm

Example Find "a" in an array of size $n \geq 2$ with 50% "a"s.

- simple solution: linear search $O(n)$

Monte Carlo algorithm

```
1 findingA_MC(A, n, k):  
2     for i = 0 to k:  
3         select random element in A until 'a' is found
```

Complexity: $O(k)$

$$P[\text{not find a at first try}] = 1/2 \implies P[\text{not find a at k-th try}] = (1/2)^k$$

$$P[\text{find a at k-th try}] = 1 - (1/2)^k$$

Las Vegas algorithm

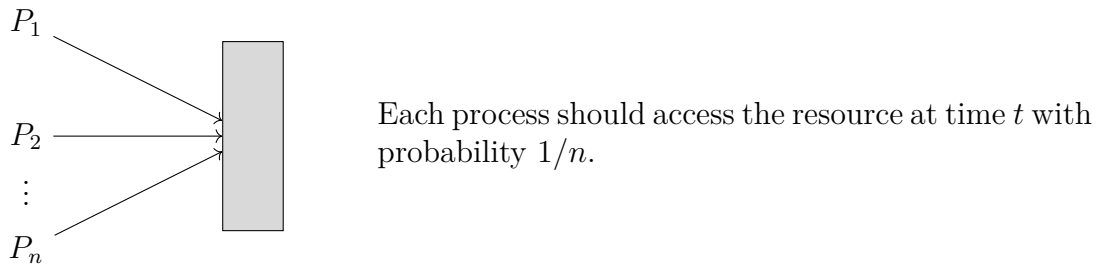
```
1 findingA_LV(A, n):  
2   while True:  
3     select random element in A until 'a' is found
```

The probability of success is 1 and the expected running time over many calls is $O(1)$, but it may run forever in the worst case.

Note: Las Vegas algorithms can be converted to Monte Carlo algorithms by setting a time limit, but not vice versa.

4.2. Contention Resolution

n processes $(p_1, \dots, p_n)$ compete for a shared resource. Simultaneous access causes collisions and failure. We want to design an algorithm for fair, regular access.



Claim. Let $S[i, t]$ be the event that process p_i successfully accesses the resource at time t , then:

$$\frac{1}{en} \leq P(S[i, t]) \leq \frac{1}{2n}$$

Proof. (not for exam)

$$P(S[i, t]) = \underbrace{p}_{\text{process } i \text{ requests}} \underbrace{(1-p)^{n-1}}_{\text{others wait}}$$

$$\text{We want } p = 1/n \implies P(S[i, t]) = (1/n) \underbrace{(1 - 1/n)^{n-1}}_{\text{between } 1/e \text{ and } 1/2}$$

$$\implies \frac{1}{en} \leq P(S[i, t]) \leq \frac{1}{2n}$$

□

Claim. The probability of failing in en rounds is at most $1/e$ and after $en(c \ln n)$ rounds is at most n^{-c}

Proof. Let $F(i, t)$ be the event that process p_i fails to access the resource in rounds 1 to t .

$$P[F(i, t)] \leq (1 - 1/en)^t \quad (1/en \leq P(S[i, t]))$$

$$\text{if } t = en \implies P[F(i, t)] \leq (1 - \frac{1}{en})^{en} \leq \frac{1}{e}$$

$$\text{if } t = en(c \ln n) \implies P[F(i, t)] \leq (1 - \frac{1}{en})^{en(c \ln n)} \leq e^{-c \ln n} = n^{-c}$$

□

Claim. The probability that all processes succeed within $2e \cdot n \ln n$ rounds is at least $1 - 1/n$.

Proof. Let $F(t)$ be the event that at least one process fails to access the resource in rounds 1 to t .

$$P[F(t)] \leq \sum_{i=1}^n P[F(i, t)] \quad (\text{union bound})$$

$$\text{if } t = 2e \cdot n \ln n \implies P[F(t)] \leq n \cdot (1/n)^2 = 1/n \quad (\text{All fail})$$

$$\implies P[\text{all succeed within } 2e \cdot n \ln n \text{ rounds}] \geq 1 - 1/n$$

□

4.3. Randomized Divide-and-Conquer

4.3.1. Quicksort

Standard quicksort

```

1 def quicksort(Array of n elements):
2     if n == 1: return
3
4     pivot = A[0] # or last element
5
6     left, right = [elements <= pivot], [elements > pivot]
7
8     quicksort(left); quicksort(right)
```

Partitioning algorithm

40	20	10	80	60	50	7	30	100
40	20	10	80	60	50	7	30	100
40	20	10	80	60	50	7	30	100
40	20	10	80	60	50	7	30	100
40	20	10	30	60	50	7	80	100
40	20	10	30	60	50	7	80	100
40	20	10	30	60	50	7	80	100
40	20	10	30	7	50	60	80	100
40	20	10	30	7	50	60	80	100
40	20	10	30	7	50	60	80	100
7	20	10	30	40	50	60	80	100

Las Vegas quicksort

```
1 def randomized_quicksort(Array of n elements):
2     if n == 1: return
3
4     pivot = A[random index] # random pivot
5
6     left, right = [elements <= pivot], [elements > pivot]
7
8     randomized_quicksort(left); randomized_quicksort(right)
```

Running time

- Best case: choose median element as pivot $\Theta(n \log n)$
- Worst case: choose max or min element as pivot $\Theta(n^2)$

We randomly choose a splitter and if it is always bigger than 25% of the elements, and smaller than 25% of the elements, the algorithm makes $\Theta(n \log n)$ comparisons.

4.4. Numerical algorithms - Buffon's needle

18th century approach to approximate π . Drop n needles randomly on a floor with parallel lines (with uniform distribution).

Theorem (Leclerc). If we drop 1 needle of length l on a floor with boards at distance d , the probability of crossing a line is: $(2 \cdot l)/\pi \cdot d$

If $l = 1$, $d = 2 \implies$ probability is $1/\pi$.

If we drop n needles, $k = n/\pi$ of them cross a line.

Bernoulli distribution It models the probability of success of a single trial, with p probability of success and $1 - p$ probability of failure.

$$P(n) = \begin{cases} 1 - p & \text{if } n = 0 \\ p & \text{if } n = 1 \end{cases}$$

The expected value of a Bernoulli distribution is p and the variance is $p(1 - p)$.

Binomial distribution It models the number of successes in n independent Bernoulli trials, with probability p of success on each trial.

The expected value of a binomial distribution is np and the variance is $np(1 - p)$.

Algorithm [Buffon] With a high number of needles n , we can estimate $\pi \approx n/k$.

Let X_i be the bernoulli variable that is 1 if the i -th needle crosses a line, 0 otherwise.

For Leclerc's theorem, with $l = 1$, $d = 2 \implies P[X_i = 1] = 1/\pi$

For Bernoulli's distribution, $p = 1/\pi$ and the variance is $p(1 - p) = (1/\pi)(1 - 1/\pi)$

Then X , the binomial variable that describes the mean of n drops is:

$$X = \sum_{i=1}^n \frac{X_i}{n} : P\left[X = \frac{k}{n}\right] = \binom{n}{k} p^k (1 - p)^{n-k}$$

with

$$P\left[X - \frac{1}{\pi} < \varepsilon\right] \geq 99\%, \quad n > \frac{1440}{\varepsilon^2}$$

Animation: <http://mste.illinois.edu/activity/buffon/>

Part II.

Distributed systems

5. Distributed Algorithms

5.1. Introduction

5.1.1. Distributed environment

It is made of autonomous entities (processes, agents) with their own private memory and computing capabilities. They interact with each other by exchanging messages, and they cooperate to solve a common problem.

- **Sequential machine:** One CPU and one memory.
- **Parallel system:** Multiple CPUs and one shared memory.
- **Distributed system:** Multiple CPUs and multiple memories.

Distributed vs Parallel

	Distributed	Parallel
Machines	Many	Unique
Memory	Separate	Shared (mutual exclusion)
Distances	Big (comm. + security problems)	Small

Moreover, in a distributed system, we have an higher tolerance to failures and we can add new nodes without stopping the system.

Every entity has: input, same code, same output

Entity behavior

$$\text{State} \times \text{Event} \longrightarrow \text{Action}$$

- Finite set of states
- Events trigger an entity: clock tick, new msg, external event (user input)
- Actions: sequence of atomic operations (cannot be interrupted and must finish)
 - Data store, compute, send msg, re(set) clock, update registers, change state, NIL

Behavior $B(x)$ set of rules

- Deterministic and complete: exactly one action for each $(\text{state}, \text{event})$ pair
- Symmetric system: all entities have the same behavior $B(x) = B(y), \forall x, y \in E$

Every system can be made symmetric

Communication

- Finite transmission delays: in absence of failures, messages are sent in a finite time
- Local orientation: split horizon into in and out neighbors
- Message ordering: FIFO

Reliability restrictions

- Guaranteed delivery: messages received uncorrupted
- Partial reliability: no future failures
- Total reliability: no failures at all

Topological restriction Strongly connected graph

Knowledge restrictions Knowledge of number of nodes, number of links, diameter (longest shortest path), etc.

Time restriction Upper bound communication delay, synchronized clocks

Usually, we will consider asynchronous systems

5.1.2. Complexity measures

1. Amount of communication: number of messages sent (bits sent)
2. Time: computation + transmission + delay

5.2. Broadcast problem - Flooding

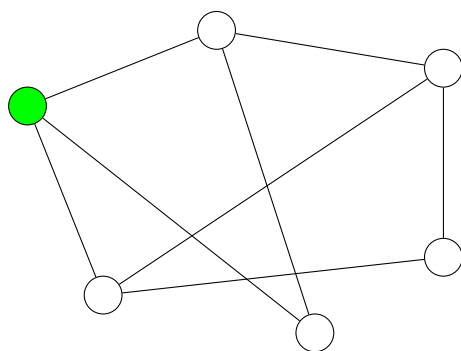
One entity wants to send its information to all the other entities of the network

Assumptions / Restrictions: Connected bidirectional graph, total reliability, unique initiator

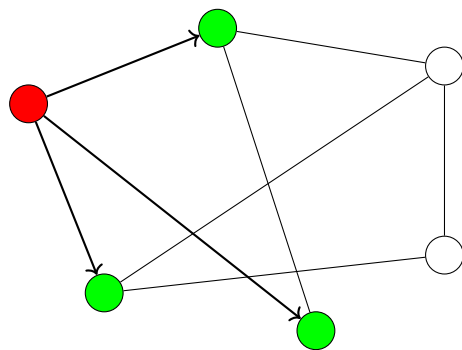
```
1 # States: {Initiator, Done}, Information I
2 # Events: spontaneously, recv(I)
3 # N(x): neighbors of x
4
5 If Initiator:
6     spontaneously:
7         send I to N(x)
8         Become done
9
10    recv(I):
11        send I to N(x) - {sender}
12        Become done
13
14 If Done:
15     do-nothing
```

Synchronous execution

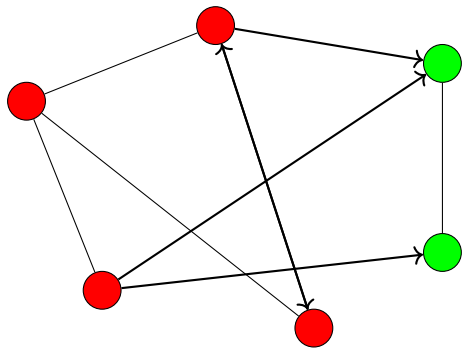
Example 1



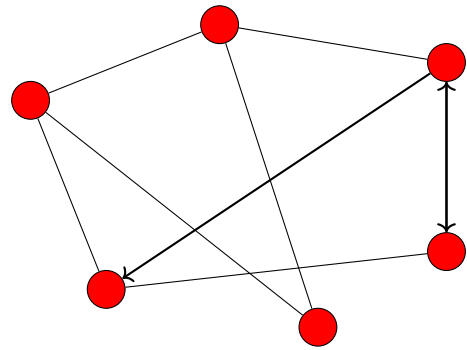
Step 1: N1 is the initiator



Step 2: N1 broadcasts to neighbors and becomes Done.

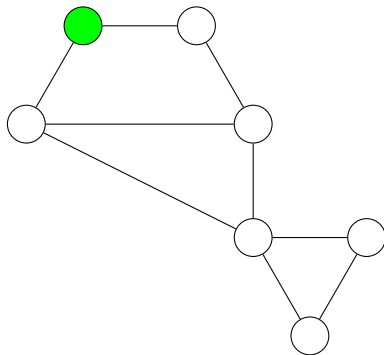


Step 3: N2, N5, and N6 propagate the message.

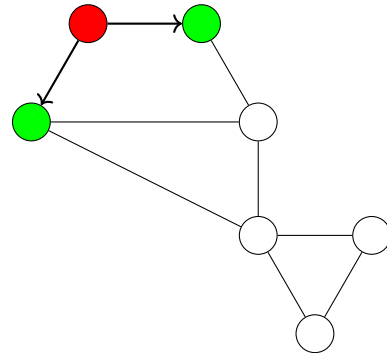


Final Step: All nodes have received the data.

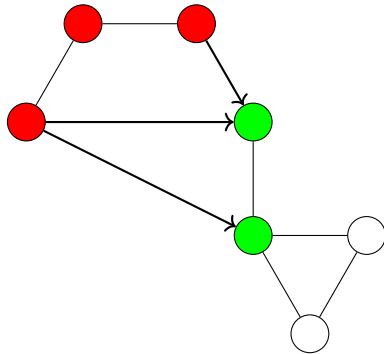
Example 2



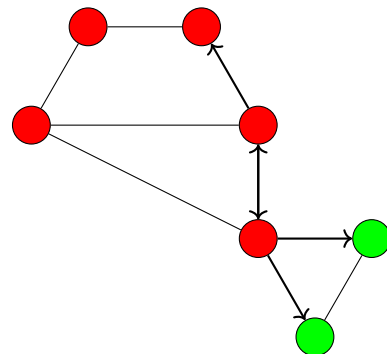
Step 1: N1 is the initiator



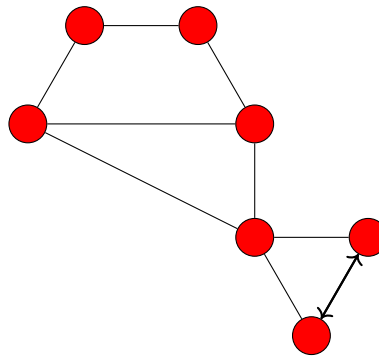
Step 2: N1 sends to neighbors and becomes Done



Step 3: N2 and N3 propagate the message

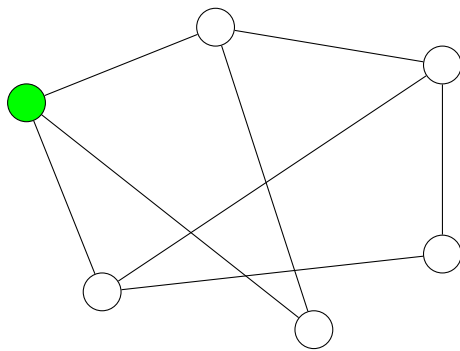


Step 4: N4 and N5 propagate to N6 and N7

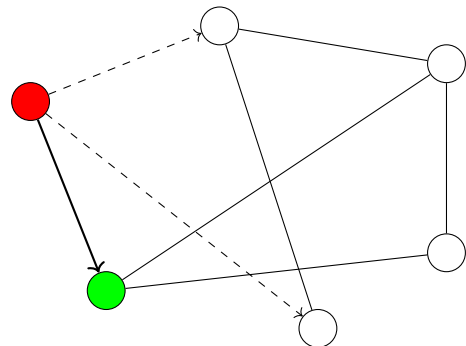


Final Step: All nodes are Done

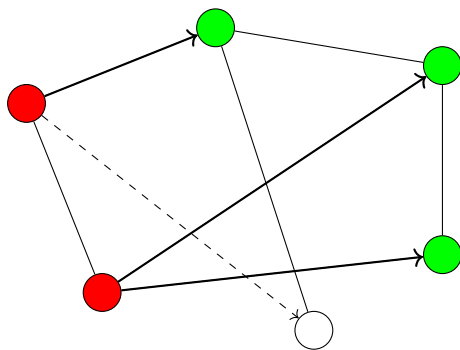
5.2.1. Asynchronous execution



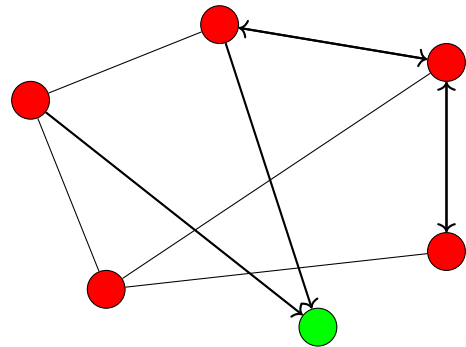
Step 1: N1 is the initiator.



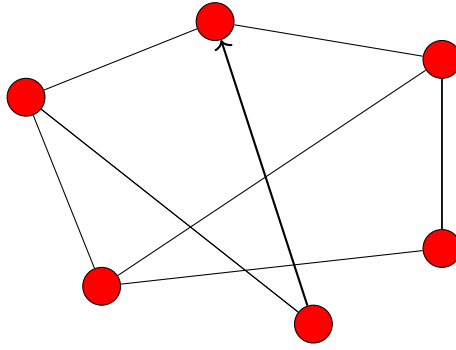
Step 2: N6 receives. Others are still pending.



Step 3: N6 propagates, N2 receives the delayed message.



Step 4: N3, N4 propagate. N5 receives the delayed message from N1 and discards the one from N2.



Final step: all nodes are Done.

Local termination: entities terminate the computation at different times

Termination detection problem: how can an entity know that all the others have terminated?

5.2.2. Correctness of a distributed algorithm

The algorithm is correct as G is connected and there is total reliability $\implies$ every entity will receive the information

The algorithm terminates as every entity receives the message and becomes done.

Worst case analysis Worst for all possible initiations and executions

Message complexity - worst case

$$\text{Messages} \leq 2 \text{ on each link} \implies \leq 2m = O(m)$$

More precisely, let s be the initiator. It will send the message to all its neighbors that will send it to all their neighbors except the sender ($N(x) - 1$).

$$|N(s)| + \sum_{x \neq s} (N(x) - 1) = \sum_x |N(x)| - \sum_{x \neq s} 1 = 2m - (n - 1)$$

Time complexity - worst case Time $\leq \text{diameter}(G) \leq n - 1 = O(n)$

Optimality Worst case, with any graph G

$$T(\text{Flooding}) \geq \text{diameter}(G)$$

$$M(\text{Flooding}) \geq n - 1$$

Flooding is time optimal, but we can use better graph topologies to reduce messages

5.2.3. N-dimensional mesh

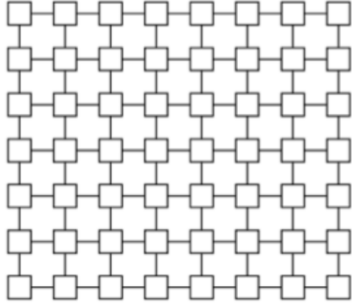


Figure 5.1.: 2D mesh

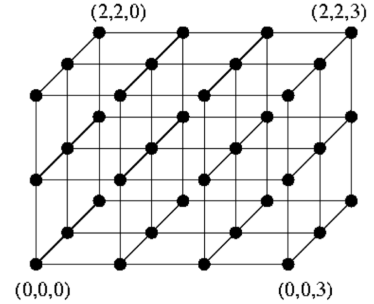


Figure 5.2.: 3D mesh

Other variations: torus, hypercube

Hypercube

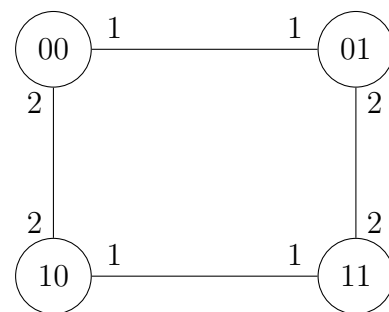
Each node does not know its own label, only the ones of the outgoing arcs.

Hypercubes of dimension k

- k bits for node labels $\implies 2^k$ nodes
- Each node has k arcs
- Edges labels indicate the bit position that differs between two connected nodes
- $m = n \cdot k/2 = (n \log n)/2 = O(n \log n)$



H1 – hypercube of dimension 1



H2 – hypercube of dimension 2

$$M(\text{flooding, hypercube}) = O(m) = O(n \log n)$$

Flooding improved for hypercubes

1. The initiator sends the message to all its neighbors
2. If you receive the message from arc labelled l , forwards it on arcs labelled $l' < l$

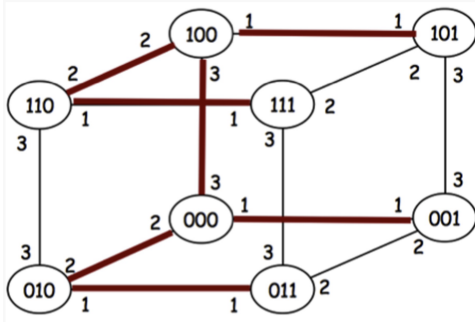


Figure 5.3.: 100 is the initiator

$$M(\text{flooding, hypercube}) = n - 1 \text{ (optimal)}$$

$$T(\text{flooding, hypercube}) = k$$

The algorithm is correct as exists a path to each node with decreasing arc labels

Complete graph If the initiator knows that the graph is complete $\left(m = \frac{n(n-1)}{2}\right)$

$$M(\text{flooding, complete}) = 2m - (n - 1) = O(n^2)$$

$$T(\text{flooding, complete}) = 1$$

Flooding improved for complete graphs

1. The initiator sends the message to all its neighbours

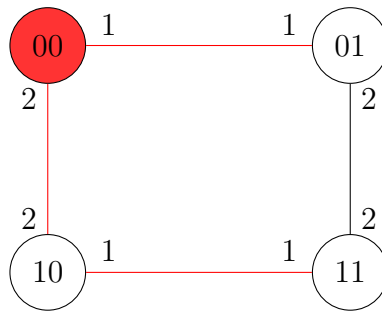
$$M(\text{flooding, complete}) = n - 1$$

$$T(\text{flooding, complete}) = 1$$

Trees We can build a spanning tree and run flooding

$$M(\text{flooding, tree}) = 2m - (n - 1) = 2(n - 1) - (n - 1) = m = n - 1$$

Hypercube example



5.3. Spanning tree construction

5.3.1. Shout protocol

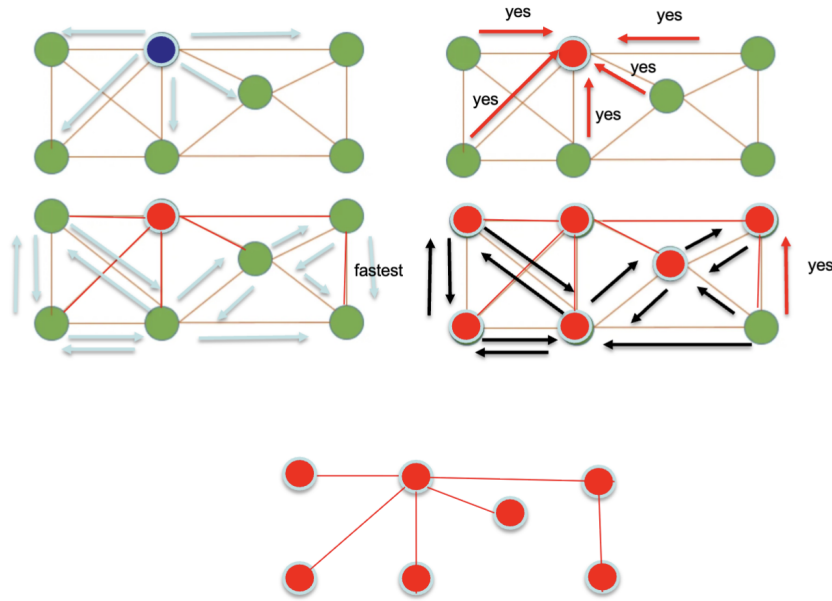
Assumptions: One initiator, connected and bidirectional graph, total reliability

```
States = {INITIATOR, IDLE, ACTIVE, DONE}
Init_States = {INITIATOR, IDLE}
Final_States = {DONE}
```

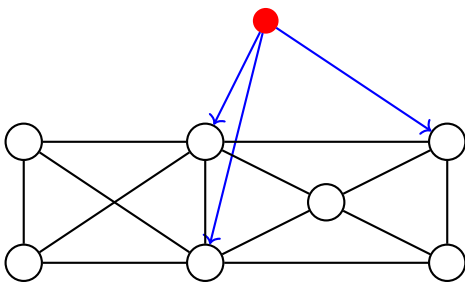
```
1 INITIATOR:
2   Spontaneously:
3     root = true, counter = 0, tree_neighbors = {}
4     send(Q) to N(x)
5     state = ACTIVE
6
7 IDLE:
8   Receiving(Q):
9     root = false, counter = 1, parent = sender
10    tree_neighbors = { sender }
11    send(yes) to sender
12    if counter == |N(x)|: state = DONE
13    else:
14      send(Q) to N(x) - { sender }
15      state = ACTIVE
16
17 ACTIVE:
18   Receiving(Q):
19     send(no) to sender
20   Receiving(yes):
21     tree_neighbors = tree_neighbors U { sender }
22     counter += 1
23     if counter == |N(x)|: state = DONE
24   Receiving(no):
25     counter += 1
26     if counter == |N(x)|: state = DONE
```

At the end, the nodes know only their tree neighbors, not the whole spanning tree.

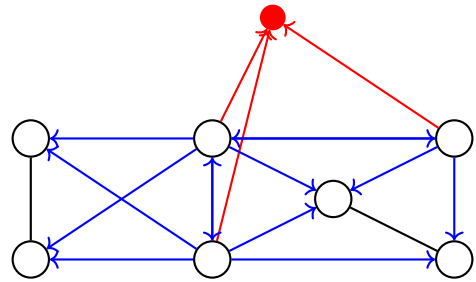
Example 1



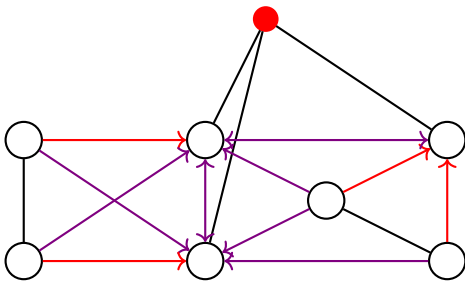
Example 2: Q, yes, no



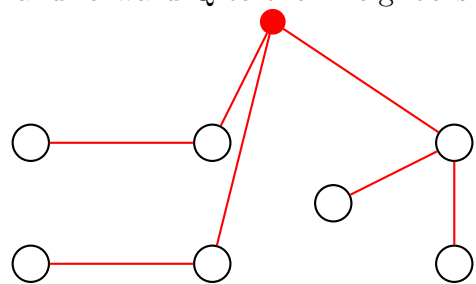
Step 1: Initiator sends Q to its neighbors



Step 2: Neighbors reply yes and forward Q to their neighbors



Step 3: Receivers send yes to one neighbor and no to the others



Final step: After forwarding other Q and replying no to duplicates, the spanning tree is built

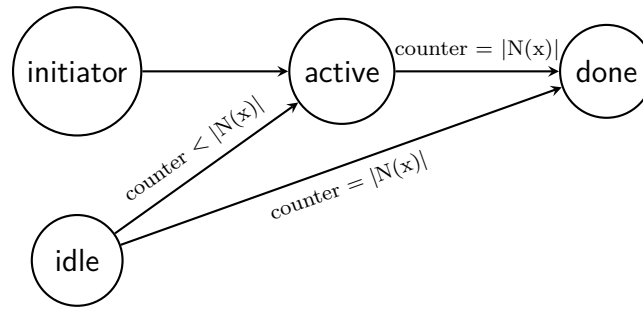


Figure 5.4.: DFA for the Shout protocol

Message complexity

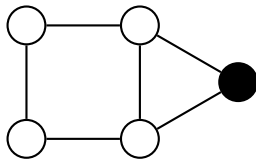
$$\text{Shout} = \text{Flood} + \text{Reply} \implies \text{Messages}(\text{Shout}) = 2 \text{ Messages}(\text{Flood})$$

Possible situations:

$Q \longleftrightarrow \text{yes}$	$2(n - 1)$
$Q \longleftrightarrow Q$	$2(m - (n - 1))$
$\text{no} \longleftrightarrow \text{no}$	$2(m - (n - 1))$

Tot: $4m - 2n + 2$

Other example



5.3.2. Shout+

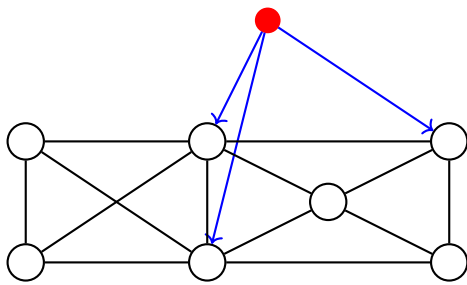
We can reduce message complexity, by ignoring duplicate Q and avoiding NOs.

```

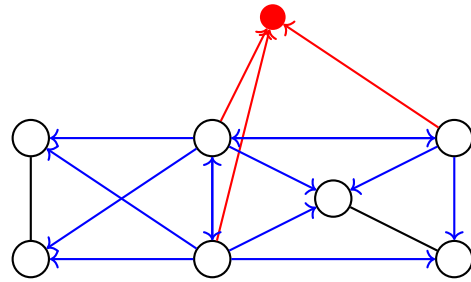
1 INITIATOR and IDLE as Shout
2
3 ACTIVE:
4   Receiving(Q): # duplicate
5     counter += 1
6     if counter == |N(x)|: state = DONE
7   Receiving(yes):
8     tree_neighbors = tree_neighbors U { sender }
9     counter += 1
10    if counter == |N(x)|: state = DONE
  
```

Messages(Shout+) = $2m$

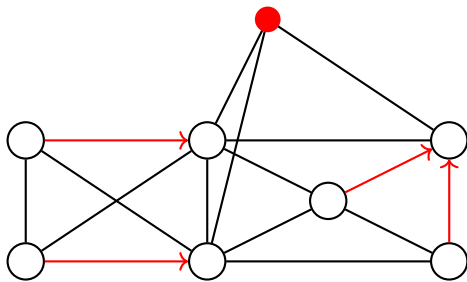
Example



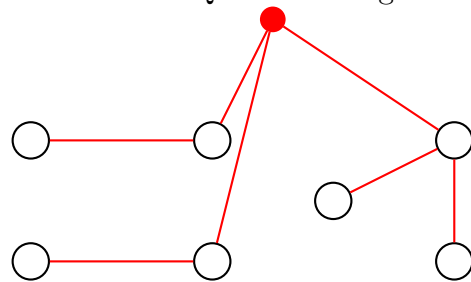
Step 1: Initiator sends Q to its neighbors



Step 2: Neighbors reply yes and forward Q to their neighbors

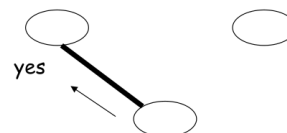
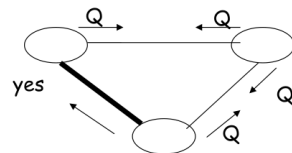
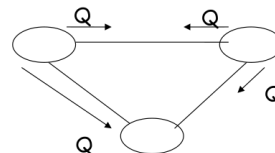
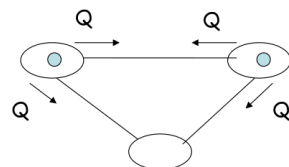


Step 3: Receivers send yes to one neighbor and ignore duplicates



Final step: After forwarding other Q, the spanning tree is built

What happens if there are multiple initiators?



We need to elect a unique initiator.

5.4. Leader election

Moving a system from an initial configuration, where all entities are **AVAILABLE** to a final configuration, where exactly one entity is **LEADER** and the others are **FOLLOWER**.

Assumptions: Bidirectional, connected graph, no failures

Theorem. The leader election problem is unsolvable unless the entities have different ids.

Proof. Assume there are two identical entities in a system. All the entities start the protocol at the same time. The two identical entities will execute the same actions and will become leaders at the same time, but the leader has to be unique.

5.4.1. Saturation Technique

Assumptions: Bidirectional tree, full reliability, multiple initiators, ordered messages, knowledge of the topology

States = {AVAILABLE, ACTIVE, PROCESSING, SATURATED}

Init_State = {AVAILABLE}

1. **Activation phase:** Initiators become **AWAKE** and wake up other nodes
2. **Saturation phase:** Leaves send saturation messages to saturate the tree
3. **Resolution phase:** Saturated nodes send resolution messages to elect a leader

Note: Different phases of the algorithm might be interleaving.

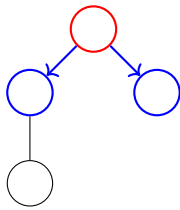
```
1 AVAILABLE:
2   Spontaneously:
3     send "activate" to N(x)
4     if |N(x)| == 1: # I'm a leaf
5       send "saturation" to parent
6       state = PROCESSING
7     else: state = ACTIVE
8
9   Receiving "activate":
10    send "activate" to N(x) - { sender }
11    if |N(x)| == 1: # I'm a leaf
12      send "saturation" to parent
13      state = PROCESSING
14    else: state = ACTIVE
15
16 ACTIVE:
```

```

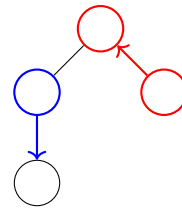
17   Receiving "saturation":
18        $N(x) = N(x) - \{ \text{sender} \}$ 
19       if  $|N(x)| == 1$ :
20           send "saturation" to parent
21           state = PROCESSING
22
23   PROCESSING:
24       Receiving "saturation":
25           state = SATURATED

```

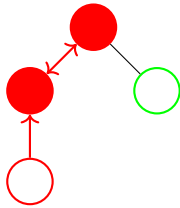
Example: synchronous execution, one initiator



Step 1: root sends wake up msgs

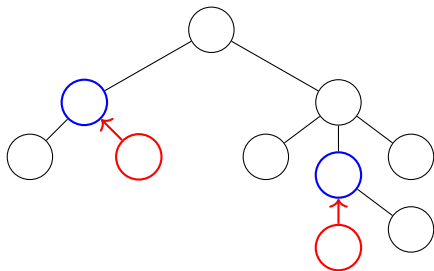


Step 2: internal node floods wak up msgs and leafs send saturation msgs

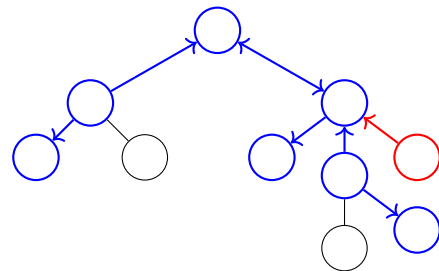


Step 3: saturated nodes send saturation msgs and an a node is elected

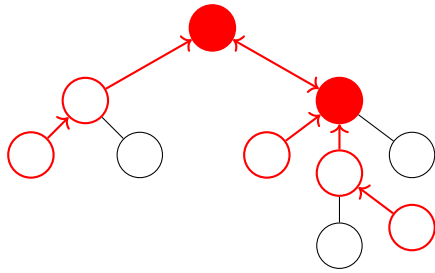
Example with multiple initiators



Step 1: some leaves wake up and send saturation msgs right away



Step 2: flooding wake up msgs. Another leaf wakes up by itself



Step 3: Sending saturation msgs and
electing a leader

Different nodes might be elected, based on the order of message delivery.

Property. Exactly two processing neighbours nodes become saturated.

Message complexity Worst case with n initiators

1. Activation: $2(n - 1)$ messages. Every node sends a wake up message to all its neighbors.

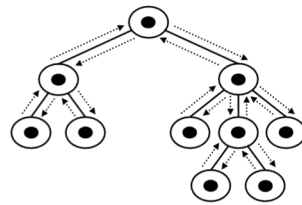
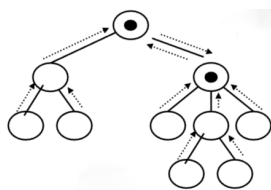
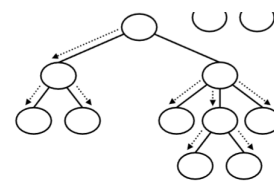


Figure 5.5.: activation

2. Saturation: $(n - 1) + 1$ messages. One saturation message per edge + 1 for the election.
3. Notification: $n - 2$ messages. Elected nodes notify their neighbors.



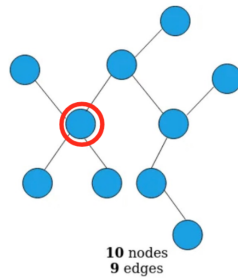
saturation



notification

Tot: $4n - 4$

Other exercise



5.4.2. Minimum finding

Each node has a value and at the end all of them must know who the minimum is.

States = {AVAILABLE, ACTIVE, PROCESSING, MINIMUM, LARGE}

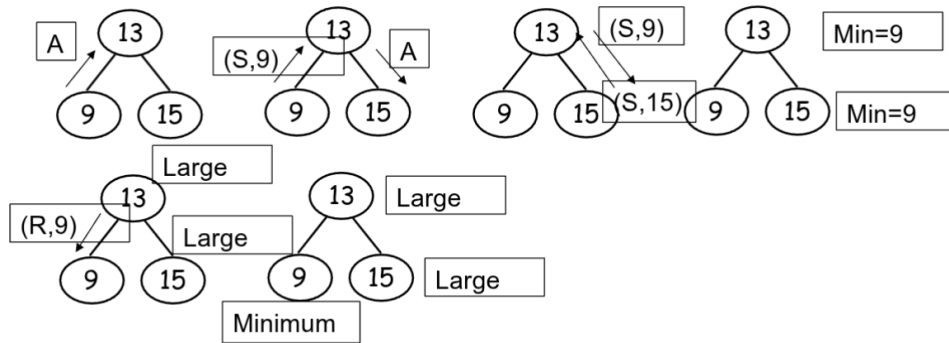
```
1 AVAILABLE:
2   Spontaneously:
3     send "activate" to N(x)
4     min = v(x) # my value
5     if |N(x)| == 1: # I'm a leaf
6       send ("saturation", min) to parent
7       state = PROCESSING
8     else: state = ACTIVE
9
10  Receiving "activate":
11    send "activate" to N(x) - { sender }
12    min = v(x) # my value
13    if |N(x)| == 1: # I'm a leaf
14      send ("saturation", min) to parent
15      state = PROCESSING
16    else: state = ACTIVE
17
18 ACTIVE:
19   Receiving ("saturation", M):
20     min = MIN(min, M)
21     N(x) = N(x) - { sender }
22     if |N(x)| == 1:
23       send ("saturation", min) to parent
24       state = PROCESSING
25
26 PROCESSING:
27   Receiving ("saturation", M): # I've been elected
28     min = MIN(min, M)
```

```

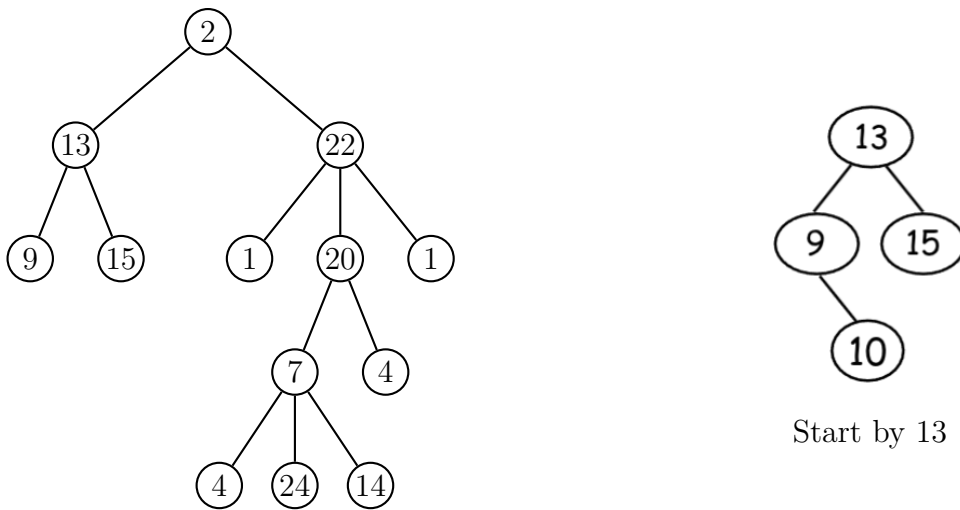
29     send ("resolution", min) to N(x) - { parent }
30     state = MINIMUM if min == v(x) else LARGE
31
32 Receiving ("resolution", M):
33     send ("resolution", M) to N(x) - { parent }
34     state = MINIMUM if M == v(x) else LARGE

```

Example 1



Other examples



Message complexity: $4n - 4$ messages. Same as saturation technique.

Message complexity with i initiators

1. Activation: $(n - 1) + (i - 1)$: wake up every node and $i - 1$ extra messages for the extra initiators.
2. Saturation and notification: $2n - 2$. Same as saturation technique

Tot: $3n + i - 4$

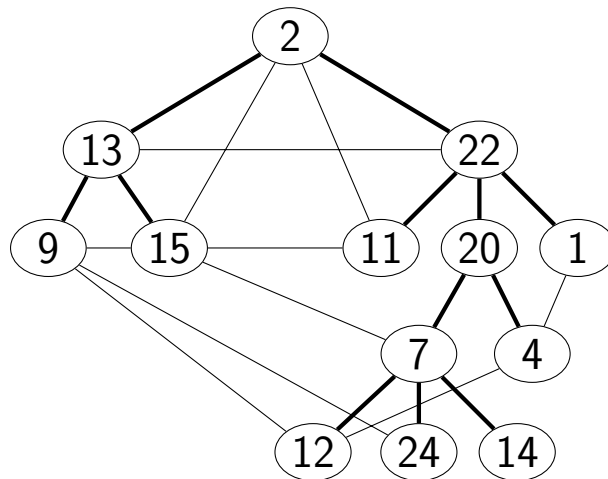
5.5. Ranking problem

Given an arbitrary graph, with different integer node numbers, find the number of smallest nodes for each node.

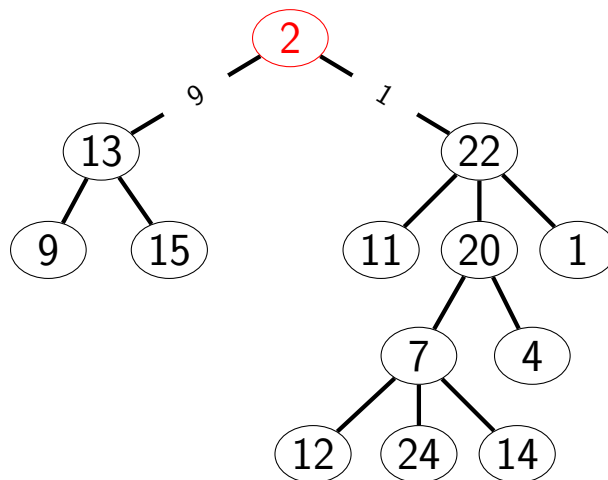
$$\text{rank}(x) = 1 + |\{y \in V : v(y) < v(x)\}|$$

Algorithm with example

1. Find a spanning tree (shout+)



2. Find a starting node using minimum finding

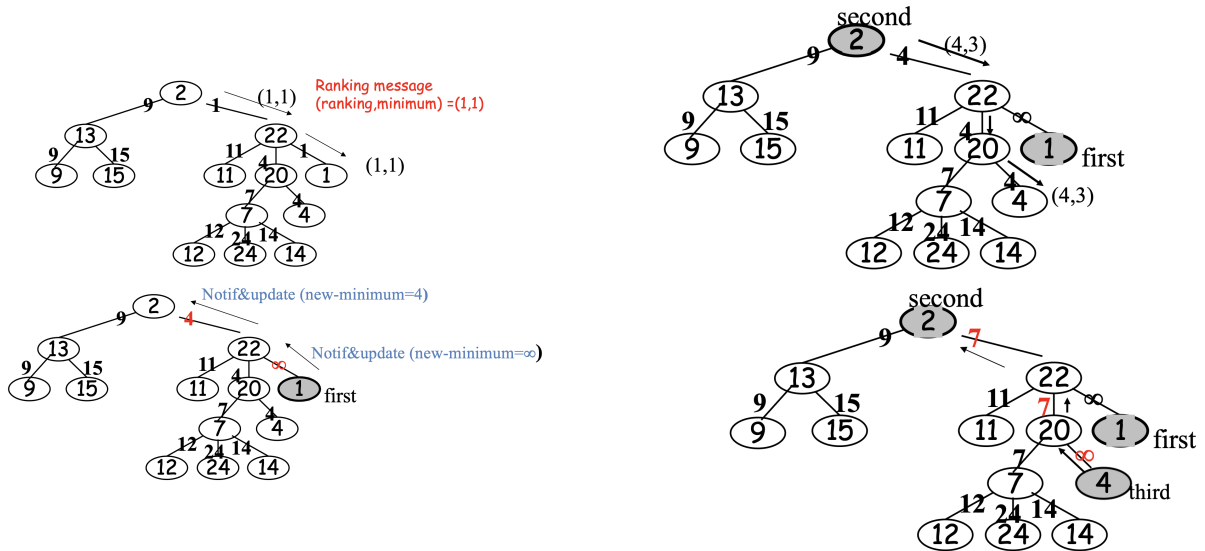


3. Ranking (centralized or de-centralized)

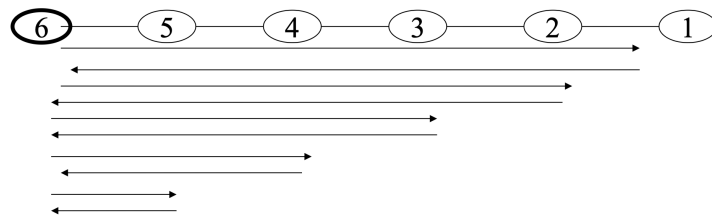
The leader (2 in this case) knows the minimum and sends a ranking message toward it.

Every other node knows the minimum in its subtrees and sends the ranking message in the right direction.

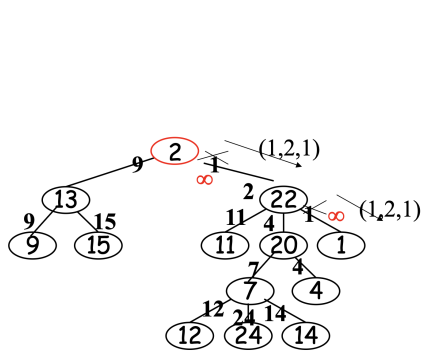
When the node receives the ranking message, it sends back to the sender an ack with the new minimum



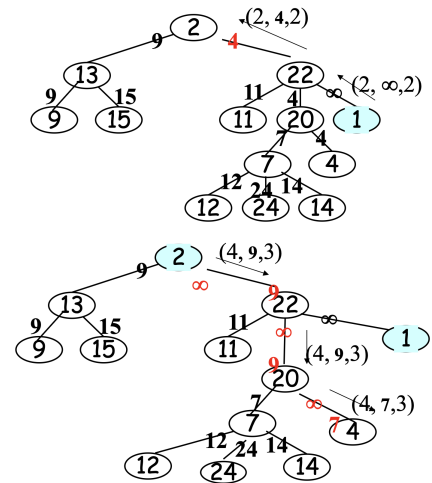
Complexity - worst case $2(n-1) + 2(n-2) + \dots + 2(1) = n(n-1) = O(n^2)$



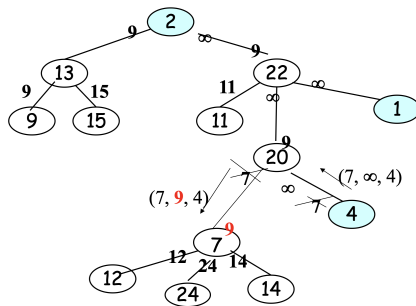
Decentralized ranking The leader sends a ranking message (first, second, rank) toward the first node. First is the smallest, second is a guess of the second smallest.



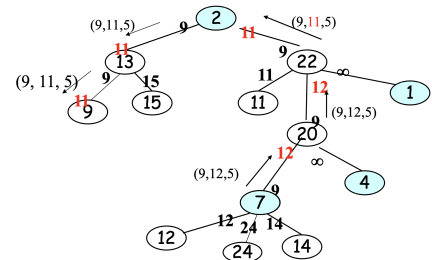
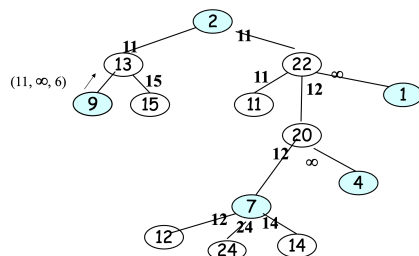
1. leader sends (1, 2, 1)



2. ranking 2nd and 3rd elements

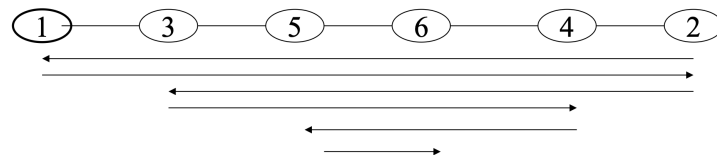


3. ranking 4th element



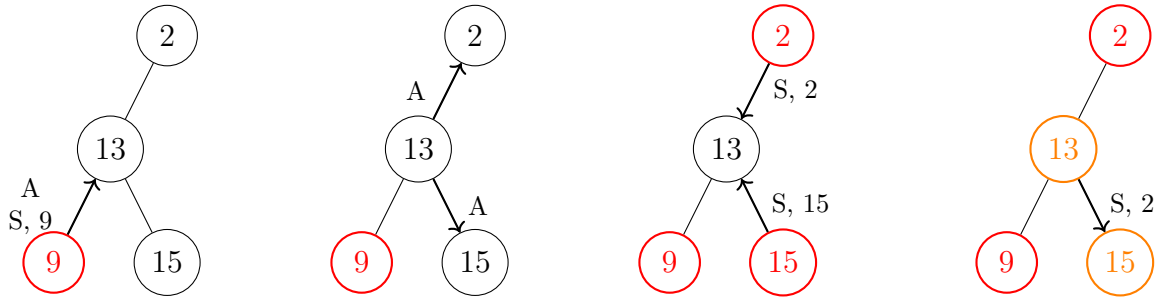
4. ranking 5th element

Complexity - worst case $2(n - 1) + (n - 2) + (n - 3) + \dots + 1 = (n - 1)(n/2 + 1)$



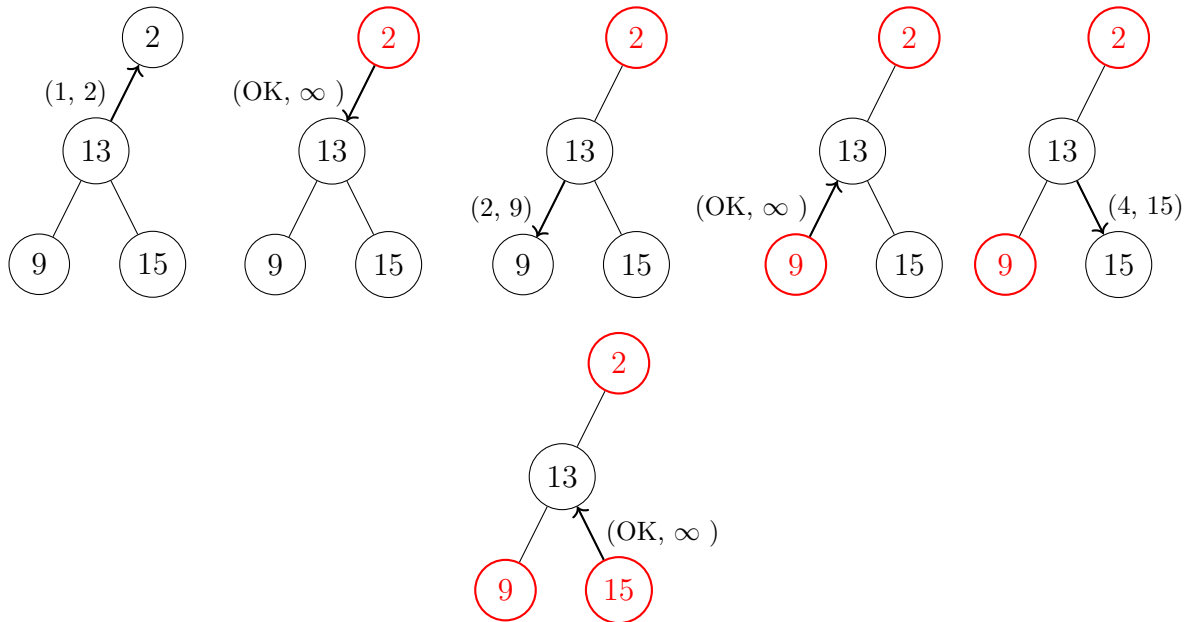
Example 1

The graph is already a spanning tree, so we elect the starting node with min finding

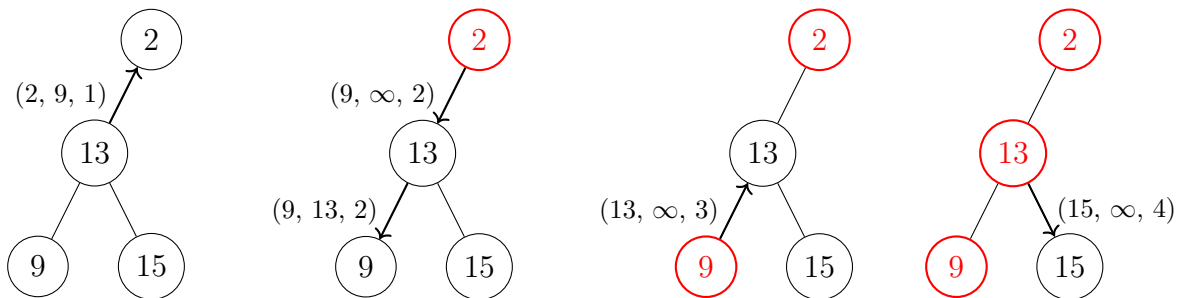


In the last step, the node 13 receives first the saturation message from 2, so at the end the elected edge is (13, 15). We elect the node with the smallest value: 13...

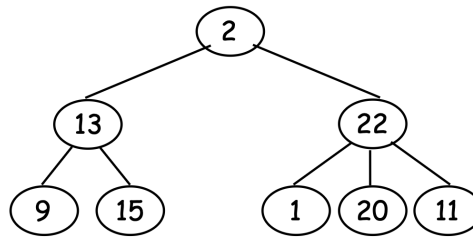
Centralized ranking:



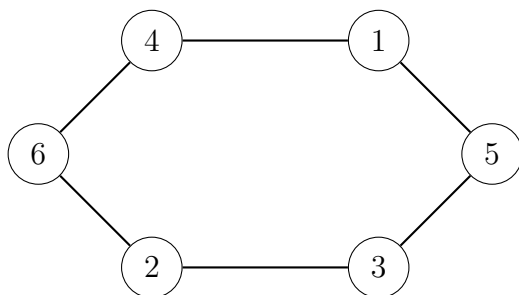
Decentralized ranking



Other example



5.6. Ring topology - Electing the minimum



n entities = m links
Each entity has two neighbors,
left and right

5.6.1. All the way

Assumptions: Local orientation and distinct ids.

Each node sends its value, forwards the received messages and keeps track of the minimum. When I receive my message back, I'm done.

Correctness and termination: We need FIFO assumption or knowledge of n , otherwise my message may overtake the others.

- Knowledge of n can be acquired with a TTL (time to live) on messages

States = { ASLEEP, AWAKE, FOLLOWER, LEADER }

S_init = { ASLEEP }

S_term = { FOLLOWER, LEADER }


```

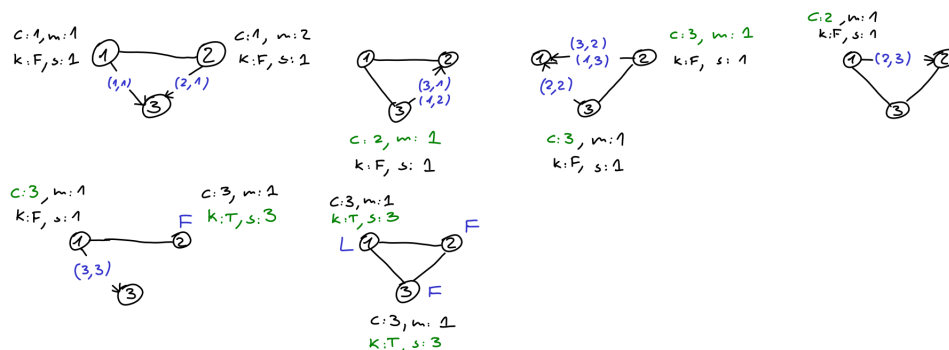
1 def Init:
2     count = 1; size = 1; known = false; min = id(x)
3     send ("Election", id(x), size) to right
4
5 def Forward:
6     send ("Election", value, ttl+1) to other
7     min = MIN(value, min); count += 1
8
9 def Check:
10    if count == size:
11        state = LEADER if min == id(x) else FOLLOWER
12
13 ASLEEP:
14     Spontaneously:          Receiving ("Election", value, ttl):
15         Init                Init
16         state = AWAKE       Forward
17                               state = AWAKE
18
19 AWAKE:
20     Receiving ("Election", value, ttl):
21         if value == id(x):
22             size = ttl; known = True
23             CHECK
24         else:
25             Forward
26             if known: CHECK

```

Complexity Each node's message visits every edge $\Rightarrow n^2$

- $M(\text{All the way}) = O(n^2 \log(\max \text{Id}))$ as a message is $\log(\max \text{Id})$ bits long
- $T(\text{All the way})$
 - Best case: n (with n initiators)
 - Worst case: $2n - 1$ (1 initiator: $n - 1$ wake up the min + n to return)

Example



5.6.2. As Far (as it can)

Same assumptions as All the way. Forward messages only if they are smaller than mine.

```

1 def Init:
2     min = id(x)
3     send ("Election", id(x)) to right
4
5 def Forward:
6     send ("Election", value) to right
7     min = value
8
9 ASLEEP:
10     Spontaneously:          Receiving ("Election", value):
11         Init                Init
12         state = AWAKE       if value < id(x): Forward
13                             state = AWAKE
14
15 AWAKE:
16     Receiving ("Election", value):    Receiving(Notify):
17         if value < min: Forward        send(Notify) to other
18         else if value == min: NOTIFY   state = FOLLOWER
19
20 NOTIFY:
21     send(Notify) to right
22     state = LEADER

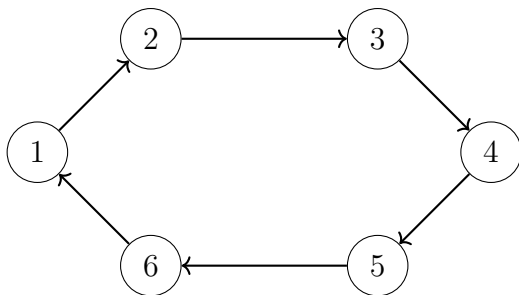
```

(In ASLEEP: Receiving I may avoid to send my own message if $id(x) > value$)

Correctness and termination: the message with the minimum will be forwarded by all nodes, returning to its owner, which will notify all the others.

Message complexity in unidirectional version

Worst case



1 $\rightarrow$ $2n$ links (forward + notify)

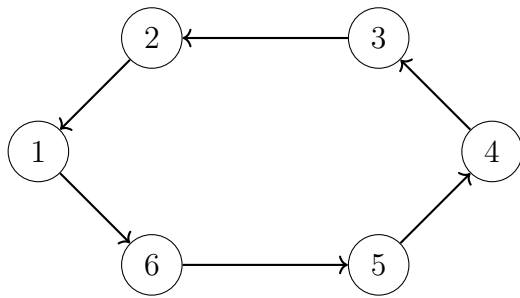
2 $\rightarrow$ $n - 1$ links

$\vdots$

$n \rightarrow$ 1 link

$$n + (n-1) + \dots + 1 = \frac{n(n+1)}{2} = O(n^2)$$

Best case



$$\begin{aligned}
 1 &\longrightarrow 2n \text{ links} \\
 \text{All the others} &\longrightarrow n - 1 \text{ links} \\
 2n + (n - 1) &= O(n)
 \end{aligned}$$

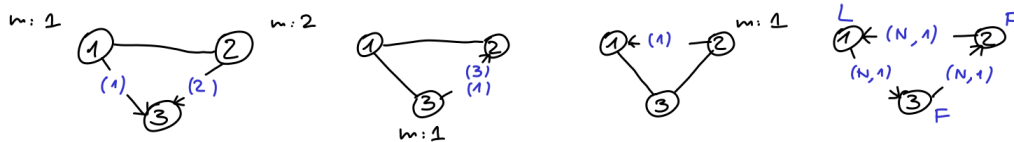
Average case: nodes are randomly ordered and the J -th smallest id average traverses n/J links

$$\sum_{J=1}^n \frac{n}{J} = n \sum_{J=1}^n \frac{1}{J} \simeq n \ln n = O(n \log n) \quad \left(\text{Harmonic series } \sum_{i=1}^n \frac{1}{i} = \ln n \right)$$

Time complexity

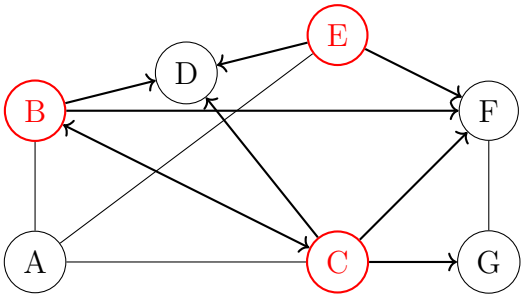
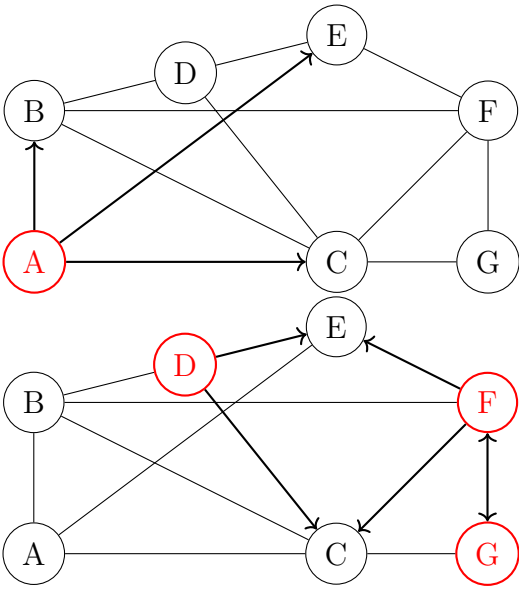
- best case: $2n$ (n initiators)
- worst case: $(n - 1) + 2n = 3n - 1$ (1 initiator: as in all the way with n more messages to notify, $n/2$ if the ring is bidirectional)

Example

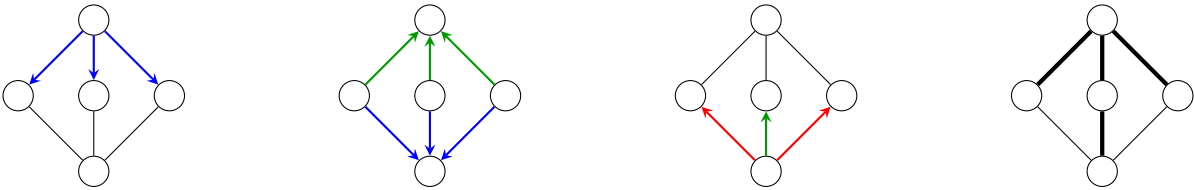


5.7. Exercises

Broadcast - synchronous execution



Shout



Saturation



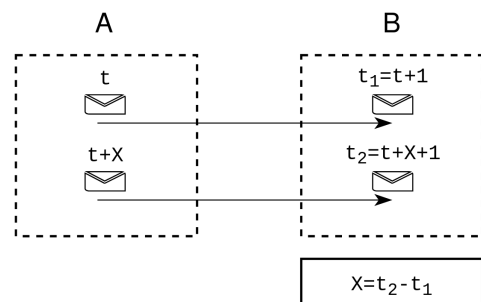
6. Synchronous Systems

k-bit communication

Sending X with 2 bits:

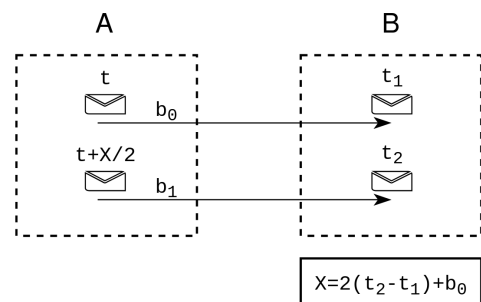
1. Send first bit
2. Wait X time units
3. Send second bit

B computes $X = t_2 - t_1$.



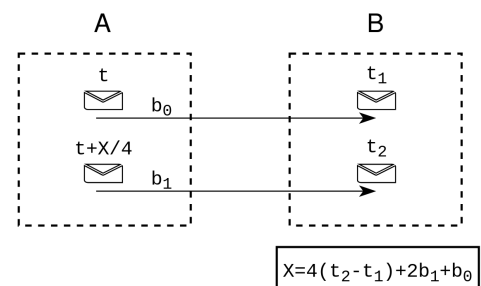
Improvement

1. Send first bit: $X \pmod{2}$
2. Wait $X//2$ time units
3. Send any second bit



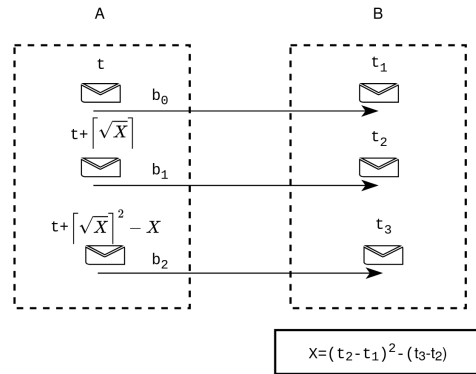
Improvement

1. Send first bit: $X \pmod{2}$
2. Wait $X//4$ time units
3. Send second bit: $X//2 \pmod{2}$



Sending X with 3 bits:

1. Send any first bit
2. Wait $\lceil \sqrt{X} \rceil$ time units and send any second bit
3. Wait $\lceil \sqrt{X} \rceil^2 - X$ time units and send any third bit



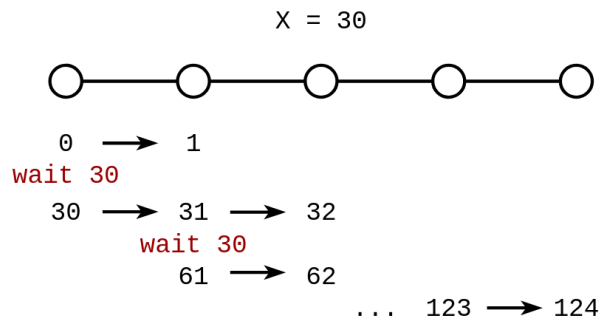
Example ($X = 40$): Send first bit, wait 7 and send a bit, wait 9 and send last bit. B receives the bits at time 1, 8 and 17 and computes

$$X = (8 - 1)^2 - (17 - 8) = 7^2 - 9 = 49 - 9 = 40$$

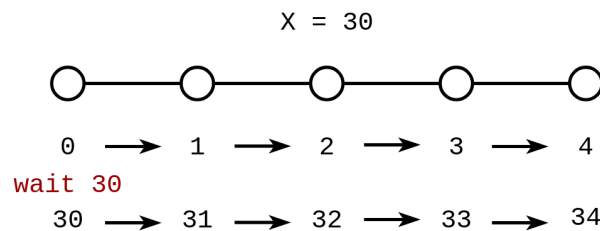
In general, with k -bit communication, we have a time complexity of $O(\lceil kX^{1/k} \rceil)$.

Pipeline technique

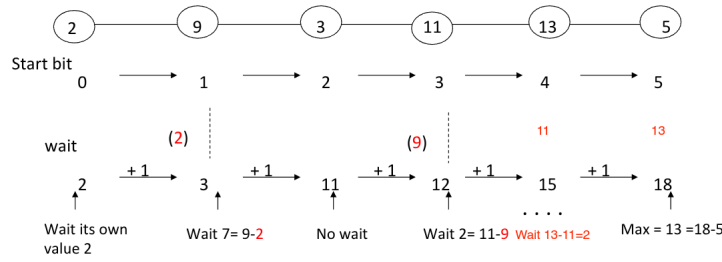
We use k -bit communication to send a message at a certain distance.



Improvement: Forward message right after receiving it, without waiting for the end bit.



Communicate the maximum



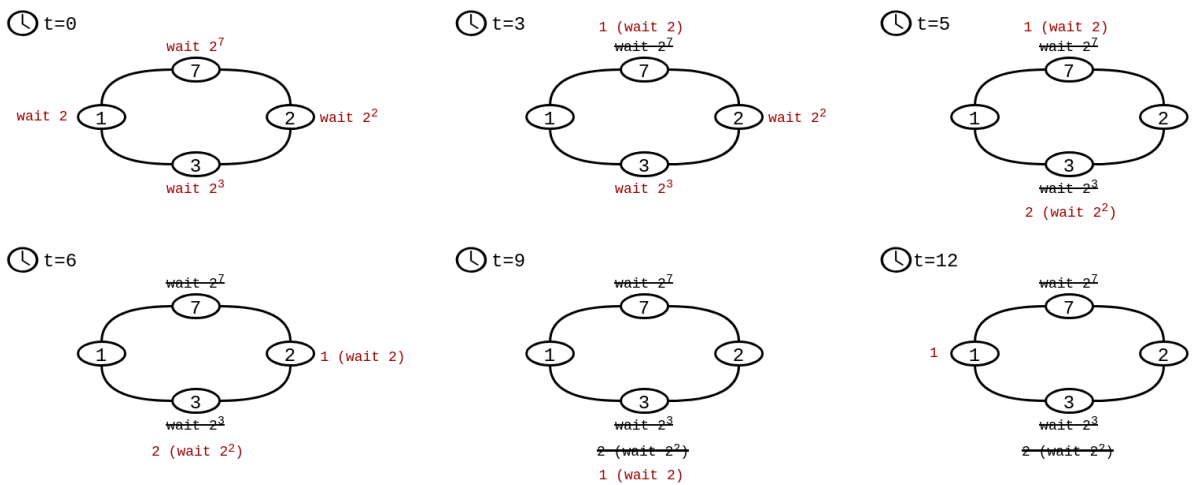
Other example: 17 - 9 - 12 - 19 - 3 (find max and notify in opposite direction)

Min finding and leader election

Idea: **Delay** messages with larger IDs and speed up those with smaller IDs, so that the minimum becomes the leader.

Assumptions: Synchronous system, unidirectional ring, no knowledge of n required.

- When a node receives a message i , it waits $f(i) = 2^i$ ticks.
- When a node receives its own message back, it becomes the leader and sends a notification message around the ring, which is not delayed.



Synchronous min finding example

The smallest ID i traverses the ring in time $2^i n + n$.

The other IDs ($i + j$, $j = 1, 2, \dots$) visit $\frac{2^i n + n}{2^{i+j}} \simeq \frac{n}{2^j}$ nodes, before being killed.

Time complexity: $O(2^i n)$

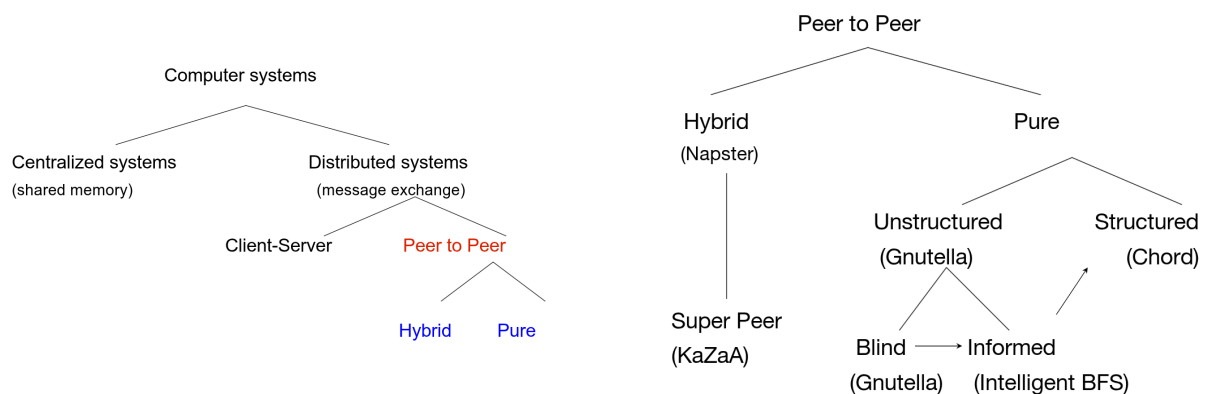
Message complexity

$$\sum_{j=1}^{n-1} \frac{n}{2^j} = n \sum \frac{1}{2^j} = O(n) \quad (\text{all the way was } O(n^2))$$

$O(n \cdot \log \text{maxId})$ considering the size of ID

7. P2P - Peer to Peer Systems

7.1. Architecture



In a client-server system, each entity has one role

- Server: provide services, managing resources
- Client: use services

The server is a single point of failure, and can be a bottleneck for the system.

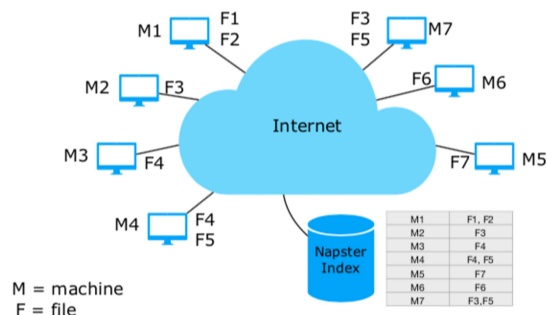
In a P2P system, there is no central server. Each entity (peer) is equal and act as both client and server at the same time. Peers can share resources and can join and leave the network at any time.

Properties: unreliable, unmanaged, uncoordinated, resilient to attacks, large collection of resources.

7.2. P2P Network Models

7.2.1. Hybrid - Napster

Napster was a music sharing service that allowed users to share their music files with each other. It was one of the first P2P systems and was very popular in the early 2000s.



- Users store their files locally

Napster architecture

Search

1. Send a request to the central server
2. The server returns a list of peers that have the file
3. The user directly connects to a peer to download the file

The central index has to handle all the queries and is still a single point of failure. Moreover, there is no guarantee about file content and quality.

7.2.2. Unstructured - Gnutella

Gnutella is a decentralized P2P system that allows users to share files with each other. It was one of the first P2P systems to be fully decentralized and was popular in the early 2000s.

Peers are equal, no matter bandwidth nor how many files you share $\implies$ no incentive to share files.

Join the network: contact a cache server and get a list of neighbors to connect to.

Search

1. Flood a query to neighbours until file is found or TTL expires (PING).
2. A peer with the file sends its address/ID to the requester (PONG).

3. The requester directly connects to a responder to download the file.

More efficient search: Iterative Deepening (start searching with low TTL and increase it until file is found or a threshold is reached), Blind, Informed, ...

Now the system is completely distributed, but flooding the network with queries is inefficient and can cause a Denial of Service attack.

7.2.3. Hybrid Super Peer - KaZaA

It combines both the features of Napster and Gnutella.

- **Super peers:** supervise a subnetwork, send queries to other super peers and can act as normal peers (as server in Napster)
- **Search:** ordinary nodes contact their own super peer. Only super peers forward queries to other super peers as in Gnutella.

Join the system: send a request to a super peer with a list of files to share

Problems How many leaves for super peer? How connect super peers (structured or not)? Adding redundancy?

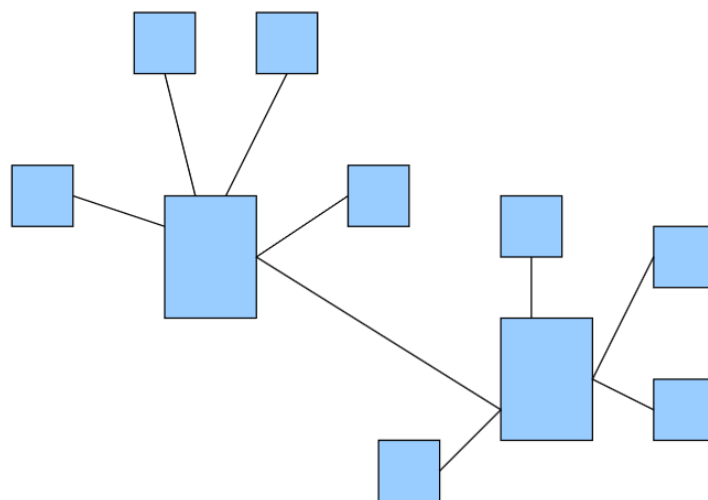


Figure 7.1.: KaZaA architecture

Disadvantages: Existing files might not be found (Gnutella) and super peers are points of failure (Napster).

Bit Torrent File sharing protocol. Peers download pieces of a file from each other, sharing already downloaded pieces (efficient fetching).

Current ways to search for files

- Centralized: $O(n)$ server storage, $O(1)$ search time (send query to server)
- Decentralized: $O(1)$ storage (TTL), $O(n^2)$ search time (flooding)

7.2.4. Structured - Chord

Scalable P2P system based on **Distributed Hash Table (DHT)** to store and retrieve files.

- **Scalability**: limit communication overhead and memory used by each node
- **Robustness and adaptability**: handle failures and changes in the network

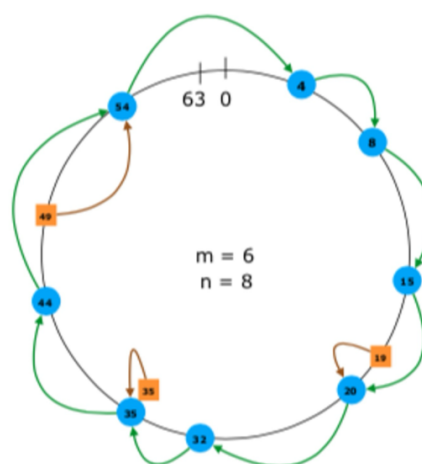
With **Distributed Hash Tables** we reach a time and space complexity of $O(\log n)$.

- Address space: peers and data mapped in the same address space, using a hash function
- Balanced and redundant: information is distributed equally among nodes and replicated to improve availability
- Adaptability: simple join and leave the network

Each peer has an ID (hash of IP) and each resource has a key (hash of title).

Join the network: we need to update all nodes' tables, generating $O(\log^2 n)$ messages.

- $\text{peerIP} \mapsto \{0, \dots, 2^m - 1\}$
- $\text{dataKey} \mapsto \{0, \dots, 2^m - 1\}$
- Each node knows its **successor**
- Data stored at the closest successor of the key



Routing strategies

Simple routing each node stores only its successor. If the data is not on a node, the next one is queried.

- **Memory:** $O(1)$
- **Search:** $O(n)$

If a node fails, the procedure is blocked.

Full routing each node stores n successors.

- **Memory:** $O(n)$
- **Search:** $O(1)$

Finger tables each node stores $\log n$ closest nodes and its predecessor.

- **Memory:** $O(\log n)$
- **Search:** $O(\log n)$

Each node knows its predecessor and its successors at distance $2^i \bmod m$ $i = 0, \dots, m$

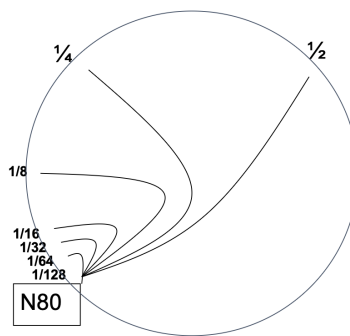
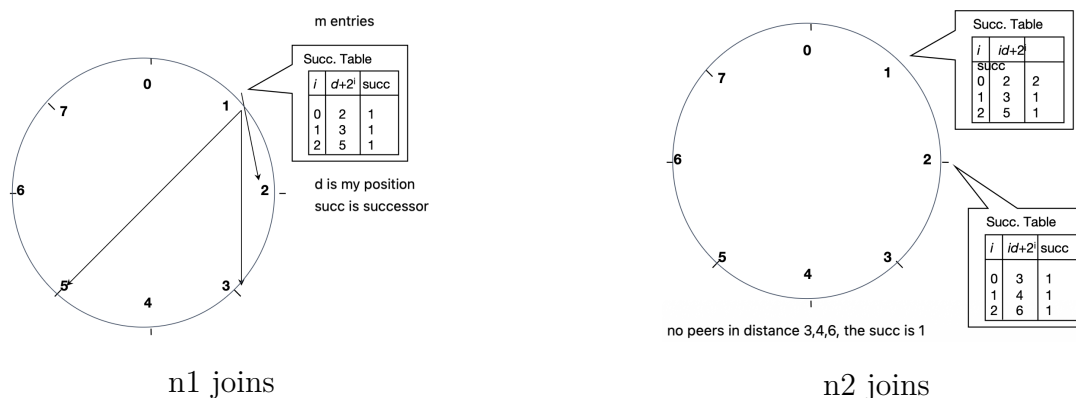
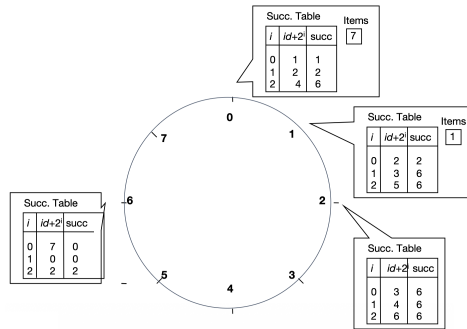


Figure 7.2.: Finger Table methodology

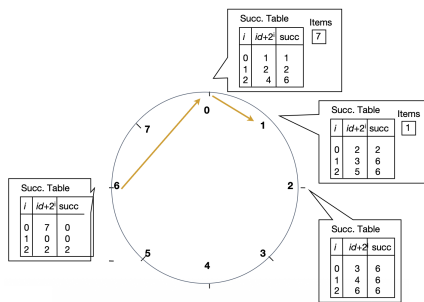
Example: $m = 3, 2^m = 8$



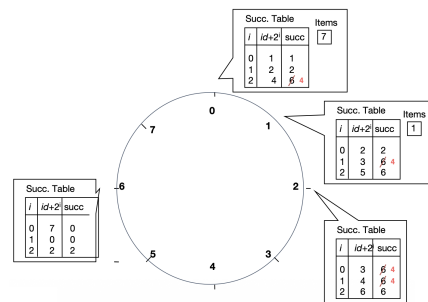


n6, n0 join. Assign items 7, 1

When we search for item k , we query the furthest known predecessor of k .



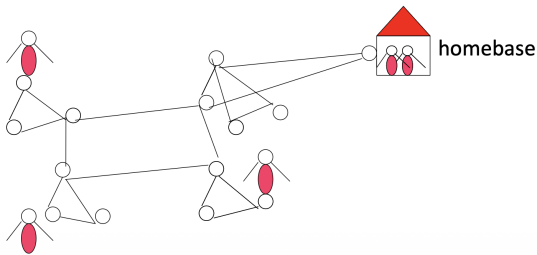
Peer 6 searches for item 1



Add peer 4. Update tables

8. Mobile agents

8.1. Model



Mobile agents have

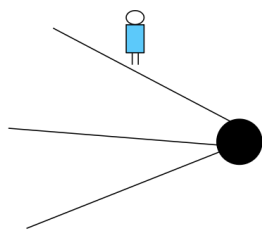
- Computing capabilities
- Local storage
- Same behavior (protocol)

Communicate using a shared whiteboard accessed in mutual exclusion

- **Synchronous setting:** agents move from one node to another in one unit of time
- **asynchronous setting:** actions require an unpredictable amount of time

8.2. Security issues

8.2.1. Harmful host - black hole

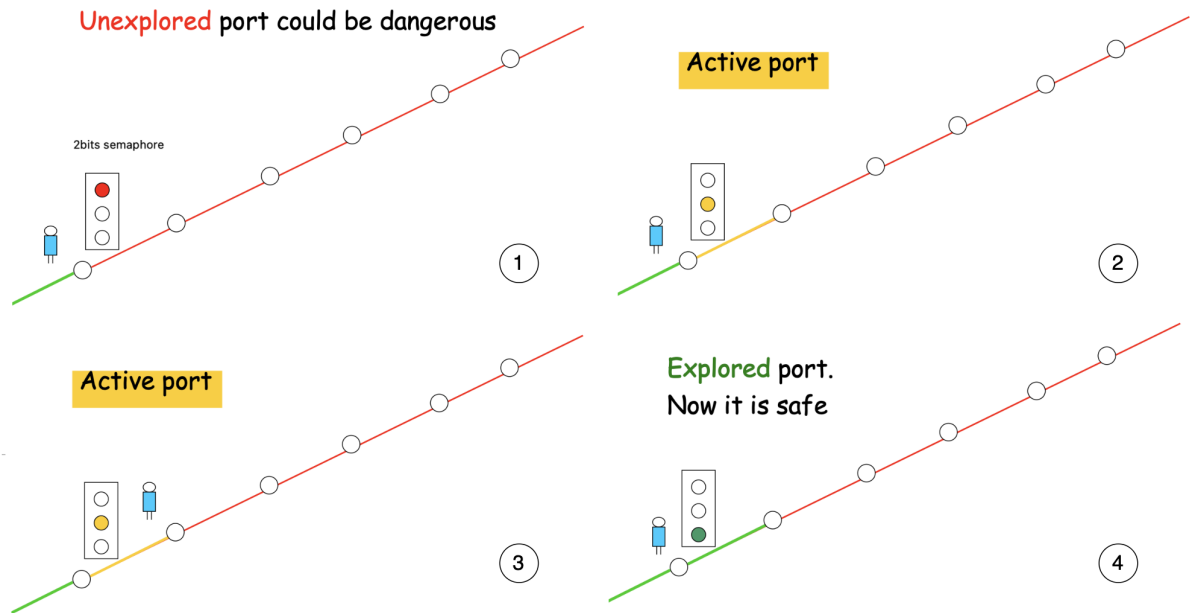


Destroy any agent that visits it, without leaving any trace.

Its location is unknown to the agents.

We want to find and report the location of the black hole (at least one agent must survive), minimizing number of agents and moves required.

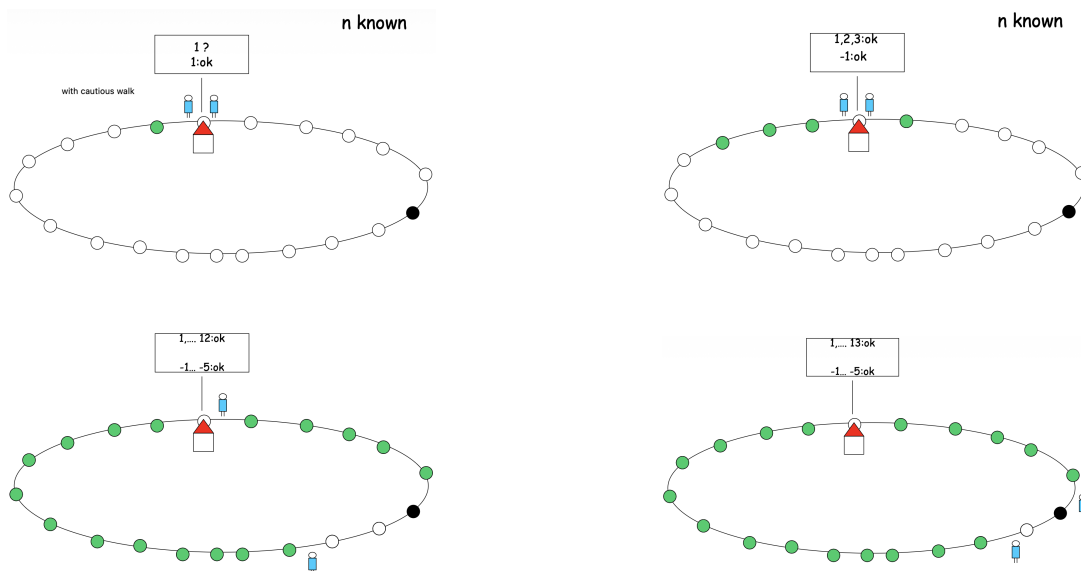
Cautious walk when an agent visits an unexplored link, set it as *active*, visit it, come back and set it as *safe*.

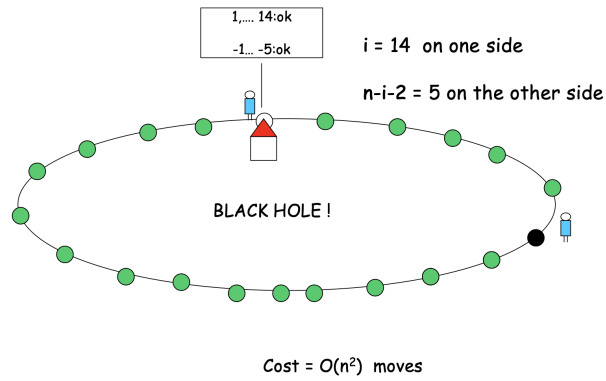


In an asynchronous setting, it is difficult to distinguish between a black hole and a slow agent.

Node-connectivity(G) ≥ 2 is necessary. Otherwise a bad agent might break the graph in two parts. (node-connectivity(Tree) = 1)

Cautious walk with ring topology (at least 2 agents required)





8.2.2. Harmful agent

It can move arbitrarily fast and it is invisible to the agents, but cannot cross a node guarded by a good agent. We want to resolve the **intruder capture problem**, which is equivalent to the **node decontamination problem**

Node decontamination problem The whole network is contaminated and an agent cleans a node by visiting it. If a node is not guarded, it becomes contaminated again.

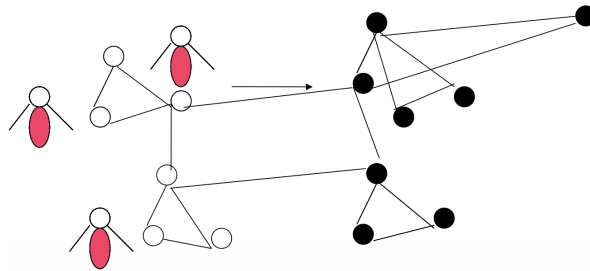


Figure 8.1.: Node decontamination problem

Contiguous monotone strategy: Clean the whole network without recontamination.

Decontaminating a mesh

- Async system, node + agent storage $O(\log n)$ bits.
- $m \times n$ mesh ($m \leq n$)

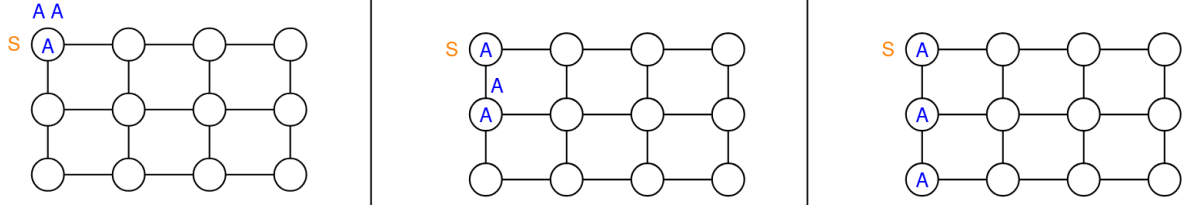
Synchronizer strategy One synchronizer with visibility power coordinates the others.

- $m + 1$ agents

Example

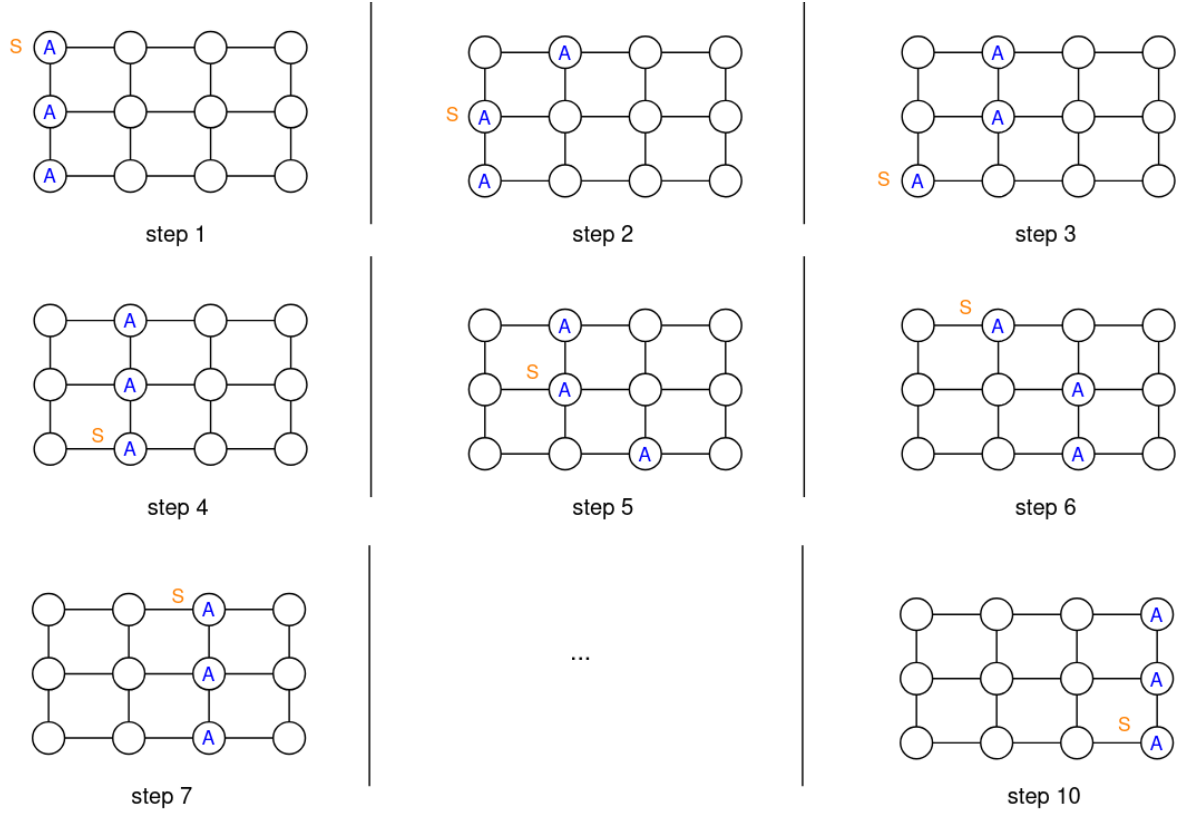
Initialization phase

- Time: $m - 1$
- Moves: $1 + 2 + \dots + m - 1 = \frac{m(m - 1)}{2}$



Cleaning phase

- Time: $(m - 1)(n - 1) + (n - 1)$ (NORTH-SOUTH + EAST)
- Moves: $m(n - 1) + (m - 1)(n - 1) + (n - 2)$ (Agents + Synchronizer)



Total

- Time: $m - 1 + (m - 1)(n - 1) + (n - 1) = mn - 1$
- Moves: $(m^2 + 4mn - 5m - 2)/2$

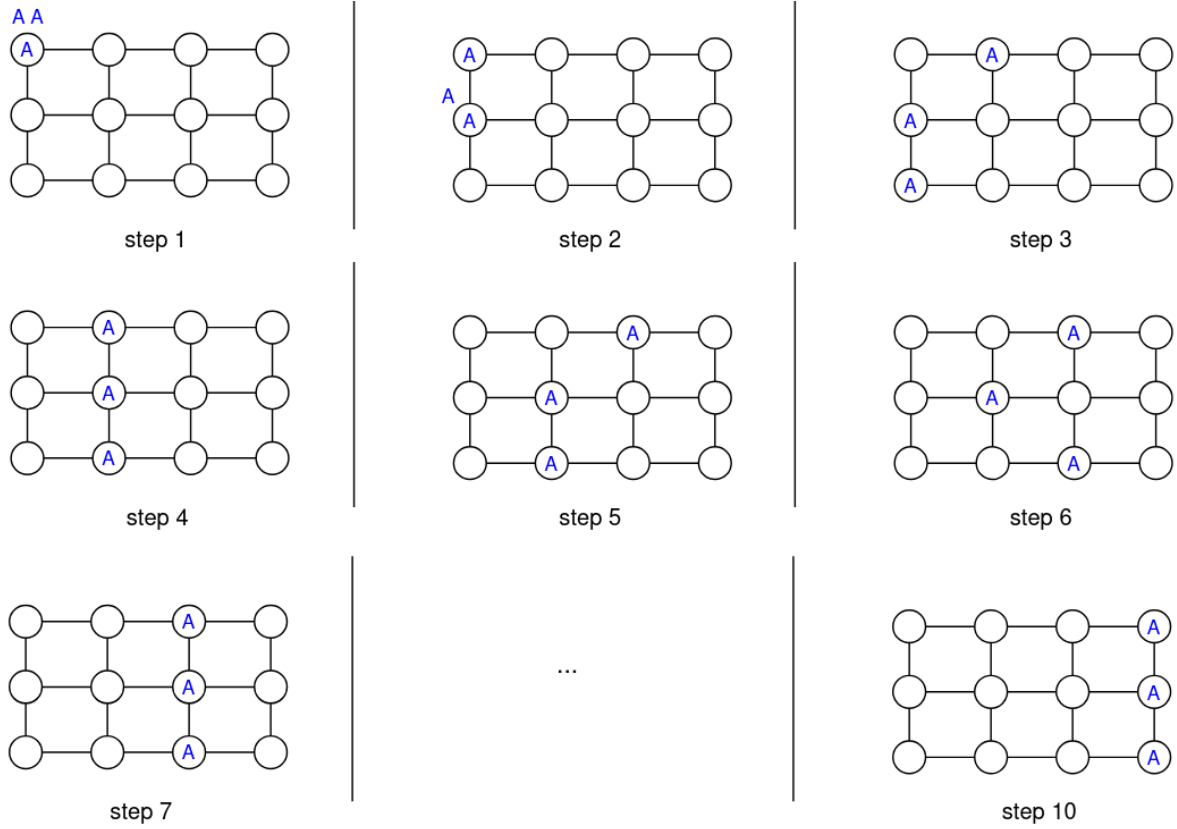
If we want to clean all the edges, we need extra m moves and $m - 1$ time units.

Agents with visibility strategy Agents move independently and can see neighboring nodes.

- m agents, no synchronizer

Example

- Time: $(n - 1) + (m - 1) ((n - 1) \text{ SOUTH}, (m - 1) \text{ EAST})$
- Moves: $\frac{m(m-1)}{2} + m(n - 1)$ (Init + Cleaning)



Lower bounds

- Time: $n + m - 2$
- Moves: $nm - 1$
- Agents: m

Summary

Search in a $m \times n$ -Mesh				
Model	Number Of	<i>Lower Bounds</i>	<i>With a Synchronizer</i>	<i>Agents with Visibility</i>
Node	Agents	m	$m + 1$	m
	Moves	mn	$\frac{m^2 + 4mn - 5m - 2}{2}$	$\frac{m^2 + 2mn - 3m}{2}$
	Time	$m + n - 2$	$mn - 2$	$m + n - 2$
Edge	Agents	m	$m + 1$	$m + 1$
	Moves	mn	$\frac{m^2 + 4mn - 3m - 2}{2}$	$\frac{m^2 + 4mn - 5m}{2}$
	Time	$m + n - 2$	$mn + m - 2$	$mn - m$

8.3. Robots

A robot is an artificially intelligent, physically embodied device that uses sensors and actuators to interact with its environment. A mobile robot specifically requires actuation to move and sensors to navigate.

Basic capabilities

- Processing and Storage: limited, execute same protocol
- Sensorial: "see" other robots, local coordinate system
- Motorial: move towards a destination

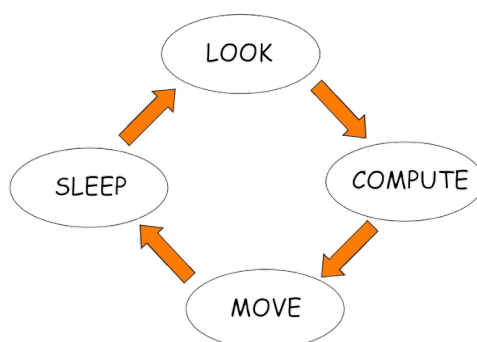


Figure 8.2.: Robot life cycle

Levels of agreement on Coordinate system: total agreement, partial agreement on axes or orientation, no agreement.

Time / Synchronization

- Fully-sync: At each clock, every robot becomes active and performs its cycle atomically
- Semi-sync: At each clock, only active robots perform their cycle atomically
- Async: No common clock, each robot performs its cycle at arbitrary times

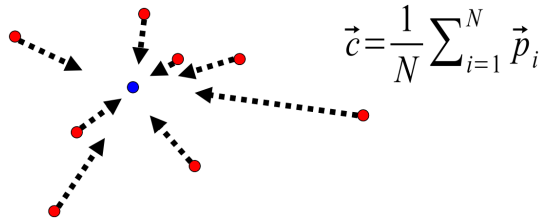
Oblivious: Robots do not remember past cycles, as they can crash or join at any time, in any state.

Communication of coordinate system: Solvable with semi-sync system

8.3.1. Gathering problem

Robots start at arbitrary distinct positions and must gather in the same place.

It is easy solvable with fully-sync system



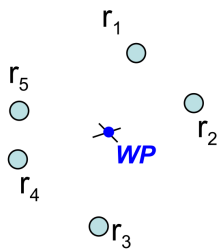
$$\vec{c} = \frac{1}{N} \sum_{i=1}^N \vec{p}_i$$

Robots meet at the **center of gravity** (barycenter)

Problem: The center of gravity can change in each round

Another solution: **Weber point**

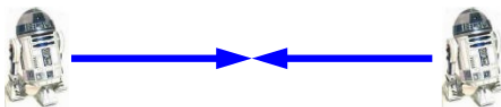
- Unique point that minimizes the sum of distances to all robots
- Invariant to robot movements



$$WP = \arg \min_{p \in \mathbb{R}^2} \sum_i \text{dist}(p, r_i)$$

Another problem: **WP** is not computable for more than 4 robots

Gathering two robots



FSYNC: go to middle point



SSYNC: convergence achievable, but not gathering

Randomization in SSYNC

Rules

1. Move to other robot's place
2. Stay in place

Algorithm

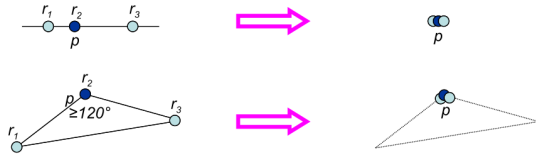
1. Flip a coin and apply one of the rules

At each round, robots gather with probability ≥ 0.5

Theorem. The SSYNC version of the gathering problem is solvable with randomization, for $n = 3, 4$

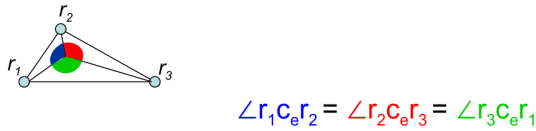
3-Gather SSYNC

Case 1: robots on the same line or form a triangle with an angle $\geq 120^\circ$



The two external robots move towards the middle one

Case 2: robots form a triangle



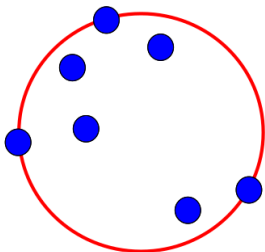
Robots move towards the center of equi-angularity c_e , invariant to robot movements

≥ 3 -Gather SSYNC

Create a single multiplicity point p^* and then gather there

Risk to avoid: the paths of two robots can intersect

Use the **Smallest Enclosing Circle** (SEC)

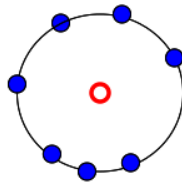


- Unique
- Computable in poly-time
- Invariant while points on it do not move

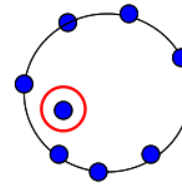
```

1 def gather():
2
3     k = robots internal to the SEC
4
5     if k == 0: robots go to SEC center
6     elif k == 1: robots go to internal robot
7
8     elif k == 2:
9         Compute Voronoi partition # one external robot per partition
10
11         if internal robots on different partitions:
12             internal robots go to SEC center
13
14         elif internal robots on same partition:
15             go to external robot of that partition
16
17         else:
18             more internal robot go to more external robot
19             more external robot go to one external robot
20
21     gather()

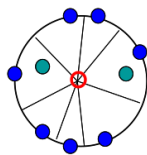
```



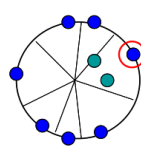
$k = 0$



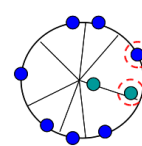
$k = 1$



$k = 2$ different partitions

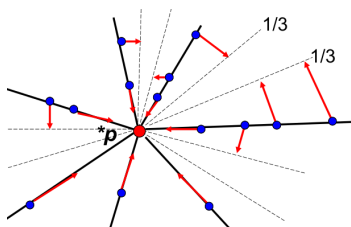


$k = 2$ same partition



$k = 2$ on boundary

Go to multiplicity point



```

1 If free path: go to p*
2 else: shift 1/3 angle to closest neighbor

```


ASync Gathering

For $N = 3, 4$ use the 3-Gather SSync algorithm

Flocking: Elect a leader and follow it

Another algorithm: Ant Colony Optimization in Swarm Intelligence in AI.

9. Bitcoin

Credit-based system Alice **trades** with Bob

- Debt to settle. It is a **risk** for Bob

Cash-based system Alice **pays** Bob

- Initial **bootstrapping** required (initial cash allocation)
- **Anonymity** and **offline transactions** possible

9.1. Bitcoin overview

Peer-to-peer network with no central authority. Transactions are registered in a public ledger (registro).

Security properties

- **Authentication:** Public key cryptography with digital signatures
- **Integrity:** Digital Signatures and Cryptographic Hash
- **Availability:** Broadcast messages to the P2P network
- **Confidentiality:** Pseudonymity

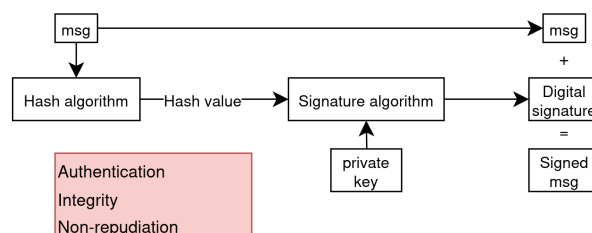


Figure 9.1.: Digital signature scheme

Note: when you buy bitcoins, you are buying a digital signature and a "slot" in the public ledger with a certain amount of money

Cryptographic hash functions are consistent, one-way and collision resistant.

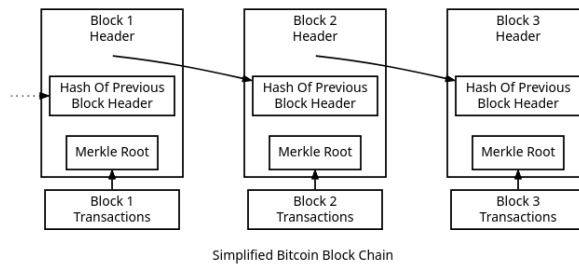


Figure 9.2.: Blockchain structure

9.1.1. Double-spending problem

Alice can spend her money twice.

Solution

- New transactions are signed and broadcasted to the network
- Miner nodes compete to find a proof of work for a new block of transactions
- The first miner to win, broadcasts the block for verification and receiving a reward
- All the nodes verify the block and add it to the chain

Proof of work: find a nonce such that the hash H starts with many zeros and $H < E$, a target value.

- $H(\text{block}): H(\text{prev_block}) || \text{nonce} || H(\text{transactions})$
- 10' limit: avoid conflicts

If two winners: the block that creates a longest chain wins. Rewards can be spent only after many blocks have been added to the chain.

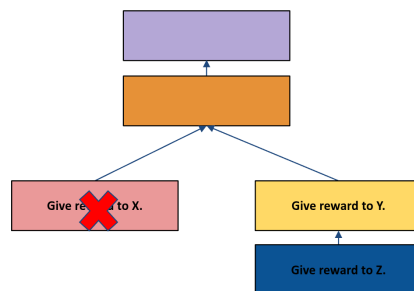


Figure 9.3.: Two winners of the proof of work